

1 Introduction

2 Relational Algebra & Relational Model

FS21, Question 1

Let us consider the first query

$$\Pi_{\text{artistName}}(\sigma_{\text{genreName}=="\text{Pop}}((\text{Artist} \bowtie \text{ReleasedBy}) \bowtie \text{Release}))$$

In a first step we need to understand what each part does

- $\Pi_{\text{artistName}}$: This is a **projection** which returns only the given attributes of a relation, in this case only the `artistName` attribute is returned for any element. In SQL terms this corresponds to only returning certain columns.
- $\sigma_{\text{genreName}=="\text{Pop}}$: This is a **selection** which based on some predicate returns elements (tuples) of some relation that satisfy the constraint given by the predicate, i.e. for them the predicate evaluates to true. Here the selection returns only tuples for which the `genreName` equals to the string "Pop".
- $\bowtie$: This is a **natural join** which joins two relations in a two step process. First we check equality on all common attributes and in a second step all duplicated common attributes are eliminated. We hence only include tuples in this relation that match on the shared attributes without duplicating said attribute, if no attributes are shared then this is equivalent to the **cartesian product** $\times$ and if all attributes are shared then this is equivalent to the **intersection** $\cap$.
 - $\text{Artist} \bowtie \text{ReleasedBy}$: The `Artist` and `ReleasedBy` relations share the attribute `artistId`, hence the result will be a relation containing the attributes `artistId`, `artistName`, `artistCountry` and `releaseId` (in this order).
 - $(\text{Artist} \bowtie \text{ReleasedBy}) \bowtie (\text{Release})$: To the above relation we now also natural join the `Release` relation to. The above relation $\text{Artist} \bowtie \text{ReleasedBy}$ will contain the `releaseId` attribute and thus share this attribute with the `Release` relation. In addition to all attributes of the above relation the resulting relation will also contain the attributes `releaseName`, `releaseDate`, `genreName`.

From the above we can retrieve that the first query first constructs a relation combining all information of releases of some artist into one relation. It then selects only the tuples that have the `genreName` as `Pop`. We then project this to a relation only containing the `artistName` and thus the result will be all names of artists that have at least one release of genre "Pop".

Let us now consider the second query

$$\Pi_{\text{userName}, \text{userSocialSecurityNumber}}(\text{User}) - \Pi_{\text{userName}, \text{userSocialSecurityNumber}}(\text{User} \bowtie (\sigma_{\text{releaseDate} > 31.12.2020}(\text{Release})))$$

Let us first again understand what each part of the query does.

- $\Pi_{\text{userName}, \text{userSocialSecurityNumber}}$: This is a **projection** which returns only the `userName` and `userSocialSecurityNumber` attributes from the relation that it is applied to.
- $\sigma_{\text{releaseDate} > 31.12.2020}$: This is a **selection** which selects only the tuples for which the `releaseDate` attribute is larger than `31.12.2020`, i.e. the days after `31.12.2020`.
- $\text{User} \bowtie (\sigma_{\text{releaseDate} > 31.12.2020}(\text{Release}))$: We first select all tuple of the `Release` relation for which the `releaseDate` attribute is larger than `31.12.2020`, i.e. all releases after `31.12.2020`. The result of the selection will have all attributes of the `Release` relation and thus it will share with the `User` relation no attribute and thus this **natural join** is equivalent to simply taking the cross product $\times$.
- $-$: This is a **difference** which returns all tuples that are in the first (left of the operator) but not in the second (right of the operator) relation. Let `UserAfterDate` be the result of $\text{User} \bowtie (\sigma_{\text{releaseDate} > 31.12.2020}(\text{Release}))$ then we are now interested in the result of

$$\Pi_{\text{userName}, \text{userSocialSecurityNumber}}(\text{UserAfterDate}) - \Pi_{\text{userName}, \text{userSocialSecurityNumber}}(\text{User})$$

We understand what `UserAfterDate` and `User` are, the projections reduce the relations only to the attributes `userName`, `userSocialSecurityNumber` and both of the relations have both of these attributes. We then take the difference of the two resulting two attribute relations.

One thing that stands out is that the natural join is equivalent to a cross product, hence we from this we cannot extract information on how the tuples inside the two relations relate to each other. They will be combined even if two tuples have nothing in common. Hence Query 2 **does not** return all the names and social security numbers of users that did not like any new release from last year, which one could simply deduce from the fact that the `UserLikesReleaseArtist` relation is not involved as well.

We can also see that the `userSocialSecurityNumber` is a primary key (it is the only key attribute) for the `User` relation and hence uniquely identifies a user and thus no two users can have the same social security number. If there were multiple key attributes then this could be possible.

The `UserLikesReleaseArtist` gives a relation between a user and a release of an artist. Because both `releaseId` and `artistId` are key attributes any release by any artist gives a different primary key together with the `userSocialSecurityNumber` hence for any release of some artist a user can have an entry in the table `userLikesReleaseArtist`.

FS21, Question 2

We are given the following tables

W	X	Y	X	Z	Z	W
1	6	a	2	c	b	2
1	2	a	2	a	b	6
4	3	b	3	e	d	6
4	4	e	5	a	e	4
5	1	e				

The tables from left to right are A, B and C. Let us look at each query and discuss what the result is.

- $A \times B \times C$: The **cartesian product** is associative in relational algebra (not in set theory however) hence we can rewrite this to $(A \times B) \times C$. We then use that for any cross product (again only in relational algebra) we have

$$|A \times B| = |A| \cdot |B|$$

and hence we get $|A \times B \times C| = A \cdot B \cdot C = (5 \cdot 4) \cdot 4 = 20 \cdot 4 = 80$ as the result.

- $(A \bowtie C) \times B$: The **natural join** of A and C has shared attributes in the attribute W. We hence only consider the rows that match the attribute W (which are only the rows containing $W = 4$) and then take the cross product of these while removing any duplicate attributes. This yields

W	X	Y	Z
4	3	b	e
4	4	e	e

We then get $|A \bowtie C| = 2$ and thus we again use the formula from above to give us $|(A \bowtie C) \times B| = |A \bowtie C| \cdot |B| = 2 \cdot 4 = 8$.

- $(A \bowtie B) \times (A \bowtie C)$: We already know the value of $|A \bowtie C|$ let us hence now consider the result of $A \bowtie B$. A and B share the attribute X and for $X = 2$ and $X = 3$ we find matching values for the attribute. We then take the cross product of the corresponding rows in the two tables, which yields the following table.

W	X	Y	Z
1	2	a	c
1	2	a	a
4	3	b	e

We hence get $|A \bowtie B| = 3$ and thus $(A \bowtie B) \times (A \bowtie C) = |A \bowtie B| \cdot |A \bowtie C| = 3 \cdot 3 = 6$.

- $(\rho_{Z \rightarrow U}(A \bowtie B)) \bowtie_{Y=Z} (\rho_{W \rightarrow V}(C))$: Let us first note that $\rho_{Z \rightarrow U}$ is a **re-naming** that renames the attribute Z of its input relation to U . The result of the query $\rho_{Z \rightarrow U}(A \bowtie B)$ is given in the table below

W	X	Y	U
1	2	a	c
1	2	a	a
4	3	b	e

The result of the query $\rho_{W \rightarrow V}(C)$ is given in the table below

Z	V
b	2
b	6
d	6
e	4

We then do a **equi-join** on the equality of the attribute Y from the first relation and the attribute Z of the second relation. This results in

W	X	Y	U
1	2	a	c
1	2	a	a
4	3	b	e

 $\bowtie_{Y=Z}$

Z	V
b	2
b	6
d	6
e	4

 $=$

W	X	Y	U	V
4	3	b	e	2
4	3	b	e	6

And thus the result is a relation with two elements, i.e. $|(\rho_{Z \rightarrow U}(A \bowtie B)) \bowtie_{Y=Z} (\rho_{W \rightarrow V}(C))| = 2$.

- $(A \ltimes B) \ltimes C$: The operator $\ltimes$ is a **left-semijoin** which works the same as a natural join but only the attributes of the first relation appear in the result. Analogous the **right-semijoin** $\rtimes$ only has the attributes of the second relation in the result. We can thus reuse the result of $A \bowtie B$ and just remove all the attributes that only appear in the relation B from the result and delete any duplicate entries (**UNION** removes duplicates by default). The result of $A \ltimes B$ can be seen below

W	X	Y
1	2	a
4	3	b

We then first natural join $A \ltimes B$ to C and remove the attributes which appear in C only. We note that $A \ltimes B$ and C share the attribute W and only the tuple $(W : 4, X : 3, Y : b)$ in the first relation and the tuple $(Z : e, W : 4)$ share the same value for the attribute.

W	X	Y
1	2	a
4	3	b

 $\bowtie$

Z	W
b	2
b	6
d	6
e	4

 $=$

W	X	Y
4	3	b

We hence get $|(A \ltimes B) \ltimes C| = 1$. As a general remark on the semijoin, a left-semijoin $\ltimes$ where R has the attributes $a_1, \dots, a_n$ is equivalent to

$$R \ltimes S = \Pi_{a_1, \dots, a_n}(R \bowtie S)$$

and a right-semijoin $\bowtie$ where S has the attributes $b_1, \dots, b_m$ is equivalent to

$$R \bowtie S = \Pi_{b_1, \dots, b_m}(R \bowtie S)$$

- $A \ltimes (B \ltimes C)$: We first compute the result of $B \ltimes C$ which can be done by first considering $B \bowtie C$ and the removing all attributes that only appear in the relation C . B and C only have one tuple each that share the same value for the shared attribute Z (e). This yields the following result

X	Z
3	e

We then take the result of this and compute the result of $A \ltimes (B \ltimes C)$ which is shown below

W	X	Y
1	6	a
1	2	a
4	3	b
4	4	e
5	1	e

 $\ltimes$

X	Z
3	e

 $=$

W	X	Y
4	3	b

And hence we conclude that $|A \ltimes (B \ltimes C)| = 1$.

- $A \bowtie (B \bowtie C)$: We can reuse the result from $B \bowtie C$ from above and instead of the attributes of B use the attributes of C in the result ($\bowtie$ is a right-semijoin), which yields

Z	W
d	6

We then compute the result of $A \bowtie (B \bowtie C)$ using the above result which yields

W	X	Y
1	6	a
1	2	a
4	3	b
4	4	e
5	1	e

 $\bowtie$

Z	W
e	4

 $=$

Z	W
e	4

Where it is important that we again delete duplicate rows due to set semantics. Bag semantics would keep both entries in the result.

- $A \bowtie \rho_{Z \rightarrow Y}(C)$: The renaming of the attribute Z of C to Y has already been done in the following result computation

W	X	Y		Y	W		W	X	Y
1	6	a		b	2		4	4	e
1	2	a	$\bowtie$	b	6	=			
4	3	b		d	6				
4	4	e		e	4				
5	1	e							

And we conclude that $|A \bowtie \rho_{Z \rightarrow Y}(C)| = 1$.

- $\Pi_Z(A \times C)$: We have already compute $A \bowtie C$ above which gave us two entries. Projecting this only two the attribute e gives us a singular row as both elements from before have $Z : e$. Hence we get $|\Pi_Z(A \times C)| = 1$
- $\rho_{W \rightarrow X}(C) \div (\Pi_X(\sigma_{W < 3}(A)))$: The operator $\div$ is a **relational division** and for $R \div S = T$ we have that T is the largest relation such that $S \times T \subseteq R$. Let us thus first compute the queries $\rho_{W \rightarrow X}(C)$ and $(\Pi_X(\sigma_{W < 3}(A)))$ and write down the results

Z	X		X
b	2	$\div$	6
b	6		2
d	6		
e	4		

By the above description we are hence looking for the largest relation T such that

<table><tr><th>X</th></tr><tr><td>6</td></tr><tr><td>2</td></tr></table>	X	6	2	×	T	=	<table><tr><th>Z</th><th>X</th></tr><tr><td>b</td><td>2</td></tr><tr><td>b</td><td>6</td></tr><tr><td>d</td><td>6</td></tr><tr><td>e</td><td>4</td></tr></table>	Z	X	b	2	b	6	d	6	e	4
X																	
6																	
2																	
Z	X																
b	2																
b	6																
d	6																
e	4																

One can see that the largest relation that allows for contains only the element $(Z : b)$ as then we get

<table><tr><th>X</th></tr><tr><td>6</td></tr><tr><td>2</td></tr></table>	X	6	2	$\times$	<table><tr><th>Z</th></tr><tr><td>b</td></tr></table>	Z	b	$=$	<table><tr><th>Z</th><th>X</th></tr><tr><td>b</td><td>2</td></tr><tr><td>b</td><td>6</td></tr></table>	Z	X	b	2	b	6	$\subseteq$	<table><tr><th>Z</th><th>X</th></tr><tr><td>b</td><td>2</td></tr><tr><td>b</td><td>6</td></tr><tr><td>d</td><td>6</td></tr><tr><td>e</td><td>4</td></tr></table>	Z	X	b	2	b	6	d	6	e	4
X																											
6																											
2																											
Z																											
b																											
Z	X																										
b	2																										
b	6																										
Z	X																										
b	2																										
b	6																										
d	6																										
e	4																										

- $\Pi_Y(A) \cap \rho_{Z \rightarrow Y}(\Pi_Z(C))$: We note that $\cap$ is the intersection and it only includes tuples that are in both relations. We first compute the queries $\Pi_Y(A)$ and $\rho_{Z \rightarrow Y}(\Pi_Z(C))$ and then compute the result, the queries have already been computed in the below result, where in both queries we removed multiple duplicate entries.

$$\begin{array}{|c|} \hline \mathbf{Y} \\ \hline a \\ \hline b \\ \hline e \\ \hline \end{array} \cap \begin{array}{|c|} \hline \mathbf{Y} \\ \hline b \\ \hline d \\ \hline e \\ \hline \end{array} = \begin{array}{|c|} \hline \mathbf{Y} \\ \hline b \\ \hline e \\ \hline \end{array}$$

and we thus conclude that $|\Pi_Y(A) \cap \rho_{Z \rightarrow Y}(\Pi_Z(C))| = 2$.

3 SQL I

3.1 FS21, Question 3

The SQL that yields the database this exercise will be about can be found in chapter 13. SQL examples can be compiled [here](#) I also use [this](#) tool on the exam database.

Exercise 1: We are given the following SQL code and are tasked to determine how many rows the query returns.

```
01 | SELECT order_status FROM orders LIMIT 15;
```

Executing this SQL query on the given database returns the following result.

order_status
4
4
3
4
2
4
4
1
4
4
4

This is a table with 11 entries, however at the exam one does not access to a SQL database, hence we must deduce this by hand. Let us do this step-by-step.

- **SELECT** : The **SELECT** statement selects columns (attributes) from some table(s) and returns them. In this case we select only the **order_status** column of all entries of the **orders** table.
- **FROM** : After this one would expect the names of the tables in the database that the columns are selected from. We find the table name **orders**.
- **LIMIT** : The **LIMIT** keyword tells us that the result of the previous query should be limited to only contain the first n entries (n is the value that comes after the **LIMIT** keyword) returned (in the ordering given, **ASC** (ascending) per default).

In this case one can see that the query below returns a table with only 11 entries.

```
01 | SELECT order_status FROM orders;
```

We hence conclude that the **LIMIT** keyword has no effect here, as it only limits the number of entries and does not mean that we must have 15 entries in the result table.

Exercise 2: We are given the following SQL code and are tasked to determine how many rows the query returns.

```
01 | SELECT * FROM products INNER JOIN categories
02 | ON categories.category_id = products.category_id
03 | WHERE categories.category_id < 4;
```

We again go through the SQL query step-by-step while building on knowledge from the previous exercise.

- **SELECT * FROM products** : The star * is a wildcard character and means that any column should be selected from the table **products**.
- **INNER JOIN** : The **INNER JOIN** keyword selects all rows from both tables as long as there is a match (based on the predicate given) between the columns that share the same attribute.
- **ON** : The **ON** keyword is used in combination with the **JOIN** keyword and it expects a predicate to come right after which we join based on.
- **WHERE** : The **WHERE** keyword is used to select only resulting rows from the previous selection based on a predicate.
- **products INNER JOIN categories ON...** : To understand what this does we first need to check what columns both tables share. The **products** table contains the columns **product_id**, **product_name**, **category_id**, **model_year** and **list_price**. The **categories** table contains the columns **category_id** and **category_names**. One can see that the two tables hence share the **category_id** column. The predicate **categories.category_id = products.category_id** after the **ON** keyword tells us that we select only rows that match in terms of the **category_id** column.

We now observe that the **products** table looks as follows.

product_id	product_name	category_id	model_year	list_price
1	Trek 820 - 2016	3	2016	379
2	Ritchey Timberwolf Frameset - 2016	3	2016	749
3	Surly Wednesday Frameset - 2016	3	2016	999
4	Trek Fuel EX 8 29 - 2016	1	2016	2899
5	Heller Shagamaw Frame - 2016	1	2016	1320
6	Surly Ice Cream Truck Frameset - 2016	3	2016	469
7	Trek Slash 8 27.5 - 2016	3	2016	3999
8	Trek Remedy 29 Carbon Frameset - 2016	3	2016	1799
9	Trek Conduit+ - 2016	2	2016	2999
10	Canyon Speedmax CFR - 2016	5	2016	12010
11	Surly Straggler 650b - 2016	4	2016	1680
12	Electra Townie Original 21D - 2016	2	2016	549
13	Electra Cruiser 1 (24-Inch) - 2016	2	2016	269
14	Trek Domane SL 6 - 2017	4	2017	3499
15	Trek Silque SLR 7 Women's - 2017	4	2017	5999

We then compare this to the `categories` table which looks as follows.

category_id	category_name
1	Comfort Bicycles
2	Electric Bicycles
3	Mountain Bicycles
4	Road Bikes
5	TT Bikes

Hence what the `INNER JOIN` in combination with the predicate `categories.category_id = products.category_id` does is to match the `category_name` attribute based on the `category_id` found. Given the first entry of the `products` table with `category_id` 3 we match it with the corresponding row in the `categories` table, i.e. the one with `category_id` 3 and `category_name` Mountain Bicycles. Doing this for every row in the `products` table yields the following table (the `category_id` column was placed duplicated such that it is easiest to see the matching rows - it would not be part of the result however).

product_id	pr..	category_id	model_year	list_price	X	category_name
1	Tr..	3	2016	379	3	Mountain Bicycles
2	Ri..	3	2016	749	3	Mountain Bicycles
3	Su..	3	2016	999	3	Mountain Bicycles
4	Tr..	1	2016	2899	1	Comfort Bicycles
5	He..	1	2016	1320	1	Comfort Bicycles
6	Su..	3	2016	469	3	Mountain Bicycles
7	Tr..	3	2016	3999	3	Mountain Bicycles
8	Tr..	3	2016	1799	3	Mountain Bicycles
9	Tr..	2	2016	2999	2	Electric Bicycles
10	Ca..	5	2016	12010	5	TT Bikes
11	Su..	4	2016	1680	4	Road Bikes
12	El..	2	2016	549	2	Electric Bicycles
13	El..	2	2016	269	2	Electric Bicycles
14	Tr..	4	2017	3499	4	Road Bikes
15	Tr..	4	2017	5999	4	Road Bikes

We then in a last step filter out all rows for which `category_id` is **not** less than 4 because of the `WHERE` keyword with the `categories.category_id < 4` predicate. This then yields the following result

product_id	pr..	category_id	model_year	list_price	category_name
1	Tr..	3	2016	379	Mountain Bicycles
2	Ri..	3	2016	749	Mountain Bicycles
3	Su..	3	2016	999	Mountain Bicycles
4	Tr..	1	2016	2899	Comfort Bicycles
5	He..	1	2016	1320	Comfort Bicycles
6	Su..	3	2016	469	Mountain Bicycles
7	Tr..	3	2016	3999	Mountain Bicycles
8	Tr..	3	2016	1799	Mountain Bicycles
9	Tr..	2	2016	2999	Electric Bicycles
12	El..	2	2016	549	Electric Bicycles
13	El..	2	2016	269	Electric Bicycles

This table has 11 rows which is the result we were looking for. We also note that depending on the SQL language used there might be a duplicated column for the `category_id` which in the above table was removed.

It however does not require this thorough argument to verify that that the output will have 11 rows, as simply seeing that we are using `INNER JOIN` (only matching rows in both tables are selected) and that exactly one row in the `products` table will match with exactly one row in the `categories` table tells us that we can simply count the number of products with `category_id < 4` which yields 11.

Exercise 3: We are given the following SQL code and are tasked to determine how many rows the query returns.

```
01 | SELECT COUNT(*) AS orders_secured, staff_id FROM orders
02 | WHERE orders.order_date < '2016-01-04'
03 | GROUP BY staff_id;
```

Let us look at the keywords we have not discussed so far.

- **COUNT(*)** : The **COUNT(...)** keyword is a function that counts the number of rows in the specified column, if ***** is used then we simply count the number of rows in the result. **COUNT** can also take an expression as input which then allows to count only rows which satisfy some condition.
- **AS** : The **AS** keyword is used to alias column names, i.e. rename the column names of some result from a query. In the above code we count the number of rows in the result from the query made and then add this as a column to the table, this is however affected by the next keyword (**GROUP BY**).
- **GROUP BY** : The **GROUP BY** keyword allows the result of the query to be split into groups according to a column. In the above example we will have for each distinct **staff_id** an entry in the result table and the **COUNT** function will count the number of rows per **staff_id** group instead of the whole table.

Let us first look at the result of the query without using the **GROUP BY** keyword.

orders_secured	staff_id
5	1

One can see that this counts the number of orders taken before date **2016-01-04**. Adding the **GROUP BY** keyword we now changes the result such that for each **staff_id** we count the number of rows in the result separately. This yields the following result. One also notices that even though there exists a **staff_id** 4 it does not appear in the result. This is because it is not selected by the initial query and thus not included in the grouping.

orders_secured	staff_id
2	1
1	2
1	3
1	5

We hence get 4 rows which is the answer to the question. One can answer this question simply by first removing all entries that have a date that does not come before **2016-01-04**, then counting the number of distinct **staff_ids** among them.

Exercise 4: We are given the following SQL code and are tasked to determine how many rows the query returns.

```
01 | SELECT products.* FROM order_items, products
02 | WHERE order_items.product_id = products.product_id
03 | UNION
04 | SELECT * FROM products
05 | WHERE list_price > 10000
```

Again, let us look at keywords not yet covered in the above exercises.

- **products.*** : This denotes all columns from the table **products** and hence we only get these columns in the result table and no columns that only appear in the **order_items** table.
- **UNION** : The **UNION** keyword takes two queries as inputs and combines their results into one table. We hence must have the same number of columns in both **SELECT** statements and the columns that are combined must have a similar data type, both columns must be ordered in the same way, i.e. either both **ASC** or both **DESC**. The default ordering is **ASC** hence if no other ordering is specified this should be satisfied. The **UNION** keyword selects only distinct rows by default, to allow duplicates one should use **UNION ALL**.

Let us first argue how one can quickly solve this. First we check that both queries have the same number of columns which the **SELECT products.*** ensures, as the second query will contain all columns of the **products** table as well. We then see that the first query selects all **products** that have been ordered at least once (the result contains one entry per distinct product that has been ordered at least once). We deduce this from **order_items.product_id = products.product_id** which will initially select all products that have been ordered, we then however restrict the result to the **products** columns and hence all duplicates are removed (again **UNION** default behaviour). This yields 11 rows for the first query as there is no order for the products with **product_id** 2, 3, 10 and 12 and there are 15 products overall.

The second query then selects all products which are more expensive than 10000, hence only a single product is found (the **Canyon Speedmax CFR - 2016**). This hence yields one row that is combined with the 11 rows from before, thus our result will contain 12 rows.

In terms of tables let us construct the query step-by-step. Let us first consider the result of the first query, i.e.

```
01 | SELECT products.* FROM order_items, products
02 | WHERE order_items.product_id = products.product_id;
```

We get the following table from the query which shows us products that have been ordered. For each time a product was ordered we expect one entry.

product_id	product_name	category_id	model_year	list_price
15	Trek Silque SLR 7 Women's - 2017	4	2017	5999
13	Electra Cruiser 1 (24-Inch) - 2016	2	2016	269
1	Trek 820 - 2016	3	2016	379
1	Trek 820 - 2016	3	2016	379
4	Trek Fuel EX 8 29 - 2016	1	2016	2899
5	Heller Shagamaw Frame - 2016	1	2016	1320
6	Surly Ice Cream Truck Frameset - 2016	3	2016	469
9	Trek Conduit+ - 2016	2	2016	2999
8	Trek Remedy 29 Carbon Frameset - 2016	3	2016	1799
1	Trek 820 - 2016	3	2016	379
4	Trek Fuel EX 8 29 - 2016	1	2016	2899
11	Surly Straggler 650b - 2016	4	2016	1680
14	Trek Domane SL 6 - 2017	4	2017	3499
14	Trek Domane SL 6 - 2017	4	2017	3499
7	Trek Slash 8 27.5 - 2016	3	2016	3999
14	Trek Domane SL 6 - 2017	4	2017	3499
15	Trek Silque SLR 7 Women's - 2017	4	2017	5999

We then look at the result of the second query, i.e.

```
01 | SELECT * FROM products
02 | WHERE list_price > 10000;
```

This yields the following table

product_id	product_name	category_id	model_year	list_price
10	Canyon Speedmax CFR - 2016	5	2016	12010

The two tables are then combined using **UNION** by sticking the result of the second query to the underside of the table of the first query and removing any duplicates.

Exercise 5: We are given the following SQL code and are tasked to determine how many rows the query returns.

```

01 | SELECT * FROM (
02 |     SELECT COUNT(*) AS sold, categories.category_name FROM products
03 |     JOIN categories ON products.category_id = categories.category_id
04 |     WHERE products.model_year > 2016
05 |     GROUP BY categories.category_id, categories.category_name
06 | )
07 | AS subquery1;

```

Let us first look at the new concepts introduced in this query.

- **JOIN** : One can write the **INNER JOIN** keywords using simply the **JOIN** keyword, as all other joins must be declared as **LEFT (OUTER) JOIN**, **RIGHT (OUTER) JOIN** and **FULL (OUTER) JOIN**, hence it always is clear that the **JOIN** keyword on its own stand for the **INNER JOIN** keyword.
- **AS subquery1** : We can not just alias column names but also an entire query result. This has no effect here as we are renaming the outer query which is never used anywhere.

Let us first look at the subquery (inner query). We first consider the following query.

```

01 | SELECT COUNT(*) AS sold, categories.category_name FROM products
02 | JOIN categories ON products.category_id = categories.category_id
03 | WHERE products.model_year > 2016
04 | GROUP BY categories.category_id, categories.category_name

```

This query will do an inner join, hence we only select rows which match on the shared columns. We can see that this simply combines the **categories** and **products** table into one table as with a natural join (which **INNER JOIN** besides the duplicates is). We then filter out all the rows that have **model_year** smaller than **2016** which leaves us with 2 rows, one for the **Trek Domane SL 6 - 2017** and one for the **Trek Silque SLR 7 Women's - 2017**. In a next step the results are grouped, we can see that both rows contain entries belonging to the **category_id 4** and the **category_name Road Bikes** hence we are left with one group. We count the number of rows belonging to said group, which gives us 2. The result of this query thus is given by the table below.

sold	category_name
2	Road Bikes

The outer query now simply takes this table and selects all columns (**SELECT * FROM**) from the table, then renames the result to **subquery1** (which has no effect on the result itself) and then returns it. Hence we are still left with the same result as before. Thus the number of rows in the result is 1.

Exercise 6: We are given the following SQL code and are tasked to determine how many rows the query returns.

```

01 | SELECT * FROM (
02 |     SELECT DISTINCT(order_id) FROM order_items
03 |     WHERE quantity <> 2
04 | ) AS subquery1
05 | LEFT JOIN orders ON orders.order_id = subquery1.order_id

```

We again first discuss the newly introduced concepts / keywords.

- **SELECT DISTINCT** : The **DISTINCT** keyword ensures that we only have per distinct `order_id` one row in the result table. We choose the first row in the result without the **DISTINCT** keyword if multiple rows are candidates.
- **<>** : The **<>** operator check if the left side is **not** equal to the right side.
- **LEFT JOIN** : The **LEFT JOIN** returns all rows from the left table and the matched records from the right table.

We again first consider the subquery.

```

01 | SELECT DISTINCT(order_id) FROM order_items
02 | WHERE quantity <> 2

```

This query simply selects any `order_id` for any order in the `order_items` table which has not been ordered exactly 2 times. One has to be a bit careful here however. Consider these two table entries of the table `order_items`.

order_id	item_id	product_id	quantity
1	1	15	1
1	2	13	2

The first row has `order_id` 1 and `quantity` 1 and hence is selected by the query. The second row has `order_id` 1 and `quantity` 2 and hence is not selected by the query. This however does not mean that `order_id` 1 will not appear in the result, as the selection happens for each row individually. In this case we thus have to consider any `order_id` that has a corresponding entry in the `order_items` table for which the `quantity` is not equal to 2. We then remove any duplicate entries (based on the `order_id` column - **DISTINCT** is being used) and hence have found the result of the subquery. It is stated below.

order_id
1
2
3
5
6
7
8
10
11

Here aliasing does have a result as it allows us to refer to the result of the subquery as `subquery1`. We then use this alias and `LEFT JOIN` the `orders` table to the `subquery1` table. We join based on if `orders.order_id` matches `subquery1.order_id` which is where the alias is used. We hence include all rows of the `subquery1` table (it is the left table) and only the ones of the `orders` table that match based on if there is a row in the `subquery1` table with the matching `order_id`.

We now see that for any row that is the result from a match between the two tables must have an `order_id` which already is in the result, hence we cannot add any non-duplicate rows. This means that the result will still only have the 9 rows of the `subquery1` table. The result look as follows (we have duplicate columns as we select any column in the result, and the `LEFT JOIN` includes the `order_id` column once for the `subquery1` table and once for the `orders` table).

order_id	order_id	customer_id	order_status	order_date	store_id	staff_id
1	1	1	4	2016-01-01	1	1
2	2	2	4	2016-01-01	2	3
3	3	3	3	2016-01-02	2	2
5	5	5	2	2016-01-03	4	5
6	6	1	4	2016-01-04	4	5
7	7	6	4	2016-01-04	3	4
8	8	3	1	2016-01-04	2	3
10	10	7	4	2016-01-05	3	4
11	11	8	4	2016-01-05	2	3

Exercise 6: We are given the following SQL code and are tasked to determine the result of the query.

```
01 | SELECT COUNT(customer_id) FROM orders, order_items
02 | WHERE orders.order_id = order_items.order_id AND (order_status < 3);
```

The newly introduced keywords are

- **AND** : The **AND** keyword is used to combine the results of two predicates into one. It returns **TRUE** if and only if both predicates evaluate to **TRUE**.

In this query we cannot simply count the number of orders in the **order** table that have **order_status < 3** as this would yield 2 which is incorrect. We must consider both predicates used with the **AND** keyword. We first find the following two entries in the **order** table with **order_status < 3**.

order_id	customer_id	order_status	order_date	store_id	staff_id
5	5	2	2016-01-03	4	5
8	3	1	2016-01-04	2	3

We now want to count the number of entries in the **order_items** table that have the same **order_id** which yields exactly three rows.

order_id	item_id	product_id	quantity
5	1	5	2
5	2	6	1
8	1	11	1

The result will simply count the number of rows and return it hence the result is 3.

Exercise 7: We are given the following SQL code and are tasked to determine the result of the query.

```
01 | (SELECT product_id FROM products WHERE list_price < 400
02 | UNION
03 | SELECT product_id FROM order_items WHERE quantity = 2)
04 | ORDER BY product_id DESC
```

The newly introduced concepts are

- **ORDER BY product_id DESC** : The **ORDER BY** keyword allows us to order a result table according to some column either in ascending (ASC) or in descending (DESC) order.

We will start with the first query.

```
01 | SELECT product_id FROM products WHERE list_price < 400
```

This will return all **product_ids** for products that cost less than 400. This is the case only for the following rows.

product_id
1
13

The second query is given by

```
01 | SELECT product_id FROM order_items WHERE quantity = 2
```

and it selects all **product_ids** for ordered items for which **quantity** is 2, i.e. all items that were ordered exactly twice. This is the case for the following **product_ids**.

product_id
13
4
5
4
14

We then take the union of the two tables while remembering that we do not keep duplicates in the results (**UNION** is **DISTINCT** by default). This selection of rows will then be ordered in descending order according to **product_id**. This produces the following table

product_id
14
13
5
4
1

Hence the correct result is 14, 13, 5, 4, 1.

Exercise 8: We are given the following SQL code and are tasked to determine the result of the query.

```
01 | SELECT customer_id AS a, COUNT(*) AS b FROM orders
02 | WHERE store_id = 1
03 | GROUP BY customer_id
04 | ORDER BY customer_id
```

Before we consider the counting and the aliasing let us first check which entries in the `orders` table have `store_id` equal to 1. These are three rows, hence arguing over these rows will be much easier. Let us look at which rows are the ones affected by count and aliasing

order_id	customer_id	order_status	order_date	store_id	staff_id
1	1	4	2016-01-01	1	1
4	4	4	2016-01-03	1	1
9	1	4	2016-01-05	1	1

We first group the rows according to `customer_id` which gives us two groups as we have two rows with `customer_id` 1 and one row with `customer_id` 4. The result is already determined now as we only have one option with two result rows, let us however still construct the result. After grouping we count the number of rows per group, then we rename the column names and also order the result by `customer_id`. This yields the following table.

a	b
1	2
4	1

Exercise 9: We are given the following SQL code and are tasked to determine the result of the query.

```
01 | SELECT categories.category_name FROM categories, products, order_items
02 | WHERE categories.category_id = products.category_id AND
03 | products.product_id = order_items.product_id
04 | GROUP BY categories.category_name
05 | ORDER BY max(list_price) ASC
06 | LIMIT 1
```

The newly introduced concepts are

- **MAX** : The `MAX(...)` function is used to compute the maximum of some column and when applied to grouped rows it is applied to each group separately.

This query seems complicated, we first need to understand what the following part of the query does.

```
01 | SELECT categories.category_name FROM categories, products, order_items
02 | WHERE categories.category_id = products.category_id AND
03 | products.product_id = order_items.product_id
```

This will select all ordered products (including duplicates - that is why we select from `order_items`). We deduce this by seeing that we match product and ordered items, the matching with `categories` is only made such that we are able to also include the `category_name` in the query result which we will group and order by. The following query was used to produce the upcoming table that may help in understanding what is done in this part of the query.

```
01 | SELECT categories.category_name, list_price, order_items.order_id, order_items.
   | item_id
02 | FROM categories, products, order_items
03 | WHERE categories.category_id = products.category_id AND
04 | products.product_id = order_items.product_id
```

This query adds some columns to the result so once can see what the result corresponds to.

category_name	list_price	order_id	item_id
Road Bikes	5999	1	1
Electric Bicycles	269	1	2
Mountain Bicycles	379	2	1
Mountain Bicycles	379	3	1
Comfort Bicycles	2899	4	1
Comfort Bicycles	1320	5	1
Mountain Bicycles	469	5	2
Electric Bicycles	2999	6	1
Mountain Bicycles	1799	7	1
Mountain Bicycles	379	7	2
Comfort Bicycles	2899	7	3

Road Bikes	1680	8	1
Road Bikes	3499	9	1
Road Bikes	3499	10	1
Mountain Bicycles	3999	11	1
Road Bikes	3499	11	2
Road Bikes	5999	11	3

We now group the result by the `category_name` column and then select the row for each group with the maximum `list_price`. This yields the following table (the `list_price` column has been added to make the result more understandable, it would not be in the result however).

<code>category_name</code>	<code>list_price</code>
Comfort Bicycles	2899
Electric Bikes	269
Mountain Bikes	379
Road Bikes	5999

We then use the `LIMIT` keyword to only select the first row of the result. This leaves us with the following result.

<code>category_name</code>	<code>list_price</code>
Comfort Bicycles	2899

It is important here to see that the fact we use the `order_items` table should tell us that we only care about ordered products. One may falsely assume that this simply retrieves the most expensive product which is not what the query does.

Exercise 10: We are given the following SQL code and are tasked to determine the result of the query.

```
01 | SELECT categories.category_name FROM categories, products
02 | WHERE categories.category_id = products.category_id
03 | GROUP BY categories.category_id, categories.category_name
04 | HAVING COUNT(*) < 3
05 | ORDER BY categories.category_id DESC
06 | LIMIT 1
```

The newly introduced concepts are

- **HAVING** : The **HAVING** keywords has the same purpose as the **WHERE** keyword. It is used in place of **WHERE** whenever we would like to combine **WHERE** with aggregate functions (**COUNT**, **MAX**, etc.) as with **WHERE** this would not work. One can hence understand **HAVING** as doing the exact same as **WHERE** but is used exactly when aggregate functions are present in the predicate.

We first notice that the **GROUP BY** keyword is used with two columns. Whenever we see **GROUP BY X, Y** it means that all rows with the same value for both **X** and **Y** belong to the same group. Hence here this does not do anything as the **category_name** is already determined by the **category_id** as **category_id** is the primary key. Let us first consider the first part of the query, i.e.

```
01 | SELECT categories.category_name FROM categories, products
02 | WHERE categories.category_id = products.category_id
03 | GROUP BY categories.category_id, categories.category_name
```

This groups all products according to their **category_id** (and thus also **category_name**) and counts the number of products per group (per **category_id**). The following table illustrates the result (cnt column was added. it is not part of the result).

category_name	cnt
Comfort Bicycles	2
Electric Bikes	3
Mountain Bikes	6
Road Bikes	3
TT Bikes	1

We then want to select the rows with count smaller than 3 (less than 3 products in that category), which requires the usage of **HAVING** as we are working with an aggregate function in the predicate. We are hence left with **Comfort Bikes** and **TT Bikes**. We then order according to the **category_id** in descending order (**DESC**). Seeing as **TT Bikes** has **category_id** 5 we are left with the following table.

category_name
TT Bikes

Exercise 11: We are given the following SQL code and are tasked to determine the result of the query.

```
01 | SELECT oi.item_id AS id, SUM(oi.quantity) AS total
02 | FROM orders AS o JOIN order_items AS oi ON o.order_id = oi.order_id
03 | WHERE o.store_id = 1
04 | GROUP BY oi.item_id
05 | ORDER BY total DESC
```

This query seems very complicated, however we once again first consider the `store_id` entries in the `orderstable` equal to 1 (the `WHERE` statement). This leaves the following three rows

order_id	customer_id	order_status	order_date	store_id	staff_id
1	1	4	2016-01-01	1	1
4	4	4	2016-01-03	1	1
9	1	4	2016-01-05	1	1

Now we consider that we use a `JOIN` (which is an `INNER JOIN` if no other keyword (`FULL`, `LEFT` or `RIGHT`) appears alongside it). We combine `orders` and `teorder_items` based on if the `order_id` matches. This allows us to combine the information at which store a order has been made with information about the quantity which both are in separate tables. For reference the following SQL query produces the upcoming table. The aliasing helps to avoid always having to refer to a table by its full name.

```
01 | SELECT * FROM orders AS o JOIN order_items AS oi ON o.order_id = oi.order_id
```

One notes that duplicate columns have been removed from the table and column names have been abbreviated to allow for the table to fit onto the page.

o.id	c.id	o.st	o.d	s.id	st.id	item_id	product_id	quantity
1	1	4	2014	1	1	1	15	1
1	1	4	2014	1	1	2	13	2
2	2	4	2014	2	3	1	1	1
3	3	3	2013	2	2	1	1	1
4	4	4	2012	1	1	1	4	2
5	5	2	2012	4	5	1	5	2
5	5	2	2012	4	5	2	6	1
6	1	4	2011	4	5	1	9	1
7	6	4	2011	3	4	1	8	1
7	6	4	2011	3	4	2	1	1
7	6	4	2011	3	4	3	4	2
8	3	1	2011	2	3	1	11	1
9	1	4	2010	1	1	1	14	2
10	7	4	2010	3	4	1	14	1
11	8	4	2010	2	3	1	7	1
11	8	4	2010	2	3	2	14	1
11	8	4	2010	2	3	3	15	1

We then only keep rows for which `store_id` is equal to 1. This leaves only the following rows.

o.id	c.id	o.st	o.d	s.id	st.id	item_id	product_id	quantity
1	1	4	2014	1	1	1	15	1
1	1	4	2014	1	1	2	13	2
4	4	4	2012	1	1	1	4	2
9	1	4	2010	1	1	1	14	2

These rows are then grouped according to `item_id`. We then have the following `SELECT` statement

```
01 | SELECT oi.item_id AS id, SUM(oi.quantity) AS total
```

Which besides renaming the `item_id` column also sums the `quantity` columns of the grouped rows. The result of this is ordered in descending (`DESC`) order which yields the following result

id	total
1	5
2	2

4 SQL II

HS21, Question 4

In this exercise we work on the same database as in Question 2. Here a view is created using the `VIEW` keyword. A view contains rows and columns, just like a real table, its fields however are from one or more real tables in the database. The database recreates the view every time a user queries it, that is because a view is more or less just a subquery with an alias which can be reused between multiple independent queries. One uses the following syntax when creating a view

```
01 | CREATE VIEW view_name AS
02 | SELECT column1, column2, ...
03 | FROM table_name
04 | WHERE condition;
```

Views are not always updatable. The following conditions make a view updatable however (all of them have to be satisfied).

- The view involves only one base relation.
- The view involves the key of that base relation.
- The view does not involve aggregate, group by, or duplicate-elimination (`UNION` for example).

As long as a view maintains a one-to-one mapping between the base-relation and its contents it is updatable.

Exercise 1: We tasked with determining what queries produce a view which is called `category_price_range` and contains the attributes `category_id`, `category_name` (both from the `categories` table), `max_price` which is the maximum `list_price` of any product of that category and `min_price` which is the minimum `list_price` of any product of that category.

Let us go through the given SQL queries one-by-one. The first option is given by

```
01 | SELECT subquery1.*
02 | FROM (
03 |     SELECT product_id, category_id, MAX(list_price) AS max_price,
04 |           MIN(list_price) AS min_price
05 |     FROM products
06 |     GROUP BY product_id
07 | ) AS subquery1
08 | JOIN categories
09 | ON categories.category_id = subquery1.category_id
```

This query does not contain the `CREATE VIEW` keyword and hence does **not** create a view. The next option is

```
01 | CREATE VIEW category_price_range AS (  
02 |     SELECT categories.category_id, categories.category_name, MAX(list_price) AS  
    max_price, MIN(list_price) AS min_price  
03 |     FROM categories, products  
04 |     WHERE categories.category_id = products.category_id  
05 |     GROUP BY categories.category_id, categories.category_name  
06 | )
```

We create a view here, we include all columns that we are asked to include, we then also group by `category_id` which ensure that `MAX` and `MIN` are computed for each of the groups separately. This hence yields the desired view.

The next option is given by

```
01 | CREATE VIEW category_price_range AS (  
02 |     SELECT category_id, MAX(list_price) AS max_price, MIN(list_price) AS min_price  
03 |     FROM (  
04 |         SELECT categories.category_id, product_id, category_name, list_price  
05 |         FROM categories, products  
06 |         WHERE categories.category_id = products.category_id  
07 |     ) AS subquery1  
08 |     GROUP BY (category_id)  
09 | )
```

We create a view, however the `category_name` column is missing, which was a requirement and we thus conclude that this does **not** create the desired view.

The last option is given by

```
01 | CREATE VIEW category_price_range AS (  
02 |     SELECT category_id, category_name, MAX(list_price) AS max_price, MIN(  
    list_price) AS min_price  
03 |     FROM (  
04 |         SELECT categories.category_id, product_id, category_name, list_price  
05 |         FROM categories, products  
06 |         WHERE categories.category_id = products.category_id  
07 |     ) AS subquery1  
08 |     GROUP BY (category_id, category_name)  
09 | )
```

This query is almost identical as the second one the only difference is that it uses a subquery to query the contents from the `categories` and `products` tables which retrieves only the desired columns. Both do the same thing however, hence we conclude that this creates the desired view.

Exercise 2: We are told to construct a query that retrieves from the view the rows for which all other products from the same category as the product itself have price lower than 3000. We first notice that the view has already grouped the product by the `category_id`, we also have access to the `max_price` per category, which tells us that all products in that category have a price at most the value of `max_price`. We are given the following SQL skeleton and are tasked to fill it out with options given to us.

```

01 | SELECT _____ A _____.* FROM
02 | _____ B _____ , (
03 |     SELECT *
04 |     FROM _____ C _____
05 |     WHERE _____ D _____
06 | ) AS _____ E _____
07 | WHERE _____ F _____

```

The options we are given are the following

```

01 | subquery1
02 | products AS P
03 | products
04 | max_price < 3000
05 | category_price_range
06 | products.category_id = subquery1.category_id
07 | min_price < 3000
08 | subquery1, products

```

We immediately eliminate the option `min_price < 3000` as this makes no sense, as the minimum price could be much lower than 3000 while many products have a price above 3000.

We then recognize that in the space E we can only place `subquery1` as reasonable option. The subquery contains a `WHERE` clause, hence we deduce that this is where we add the `max_price < 3000` to retrieve the wanted categories from the `category_price_range` view. We hence insert into C the option `category_price_range` and into D the option `max_price < 3000`.

This leaves us with A, B and F still undetermined. We still have a `WHERE` clause after the outer query and the only other option left is `products.category_id = subquery1.category_id` and hence we insert it into F. This however also implies that we select the needed rows (`products.category_id`) from the `product` table at some point, which is why we insert into A the option `products` to select all columns of the `products` table (notice the `.*`). We hence only have B to fill. Seeing as we have the option to use fields multiple times it makes sense to select the columns of the `products` table from the `products` table, hence we insert into B the option `products`.

This produces the following SQL query

```

01 | SELECT products.* FROM
02 | products , (
03 |     SELECT *

```

```
04 |      FROM category_price_range
05 |      WHERE max_price < 3000
06 |    ) AS subquery1
07 |    WHERE products.category_id = subquery1.category_id
```

Exercise 3: We are given a set of questions and are tasked to determine whether they are true or false.

Let us go through the given statements.

- Modifying the tables that were utilized in creating a view might also change the outputs of queries that utilize that view.
 - This is correct and the reasoning is that a view is created every time it is queried hence updates to the underlying tables always affect the view.
- Views are an abstraction that provide us with logical data independence.
 - This is correct because we have seen in the lecture that a relation gives us physical data independence whereas a view allows us to raising the abstraction level even further and providing us with logical data independence.
- Rows with **NULL** values do not influence the result of **COUNT(*)** (i.e., the result is the same whether or not these rows are in the input).
 - Just because a row contains **NULL** values it does not mean that all columns are **NULL** especially in well-defined database any entry should at least have the primary-key **NOT NULL**, hence the rows are also counted.
- Any view that meets the following requirement is updatable: The view contains only **ONE** base relation.
 - Not all three requirements from above are covered hence this does single condition does not suffice to make a view updatable.

5 SQL III

5.1 HS21, Question 5

We are given 8 queries and are tasked to determine what each does, or at least we should be able to determine which descriptions of an SQL query matches the given SQL query. Let us go through each query and determine what it does.

Query 1: We are given the following SQL query

```

01 | SELECT id, cnt
02 | FROM (
03 |     SELECT s.staff_id AS id, COUNT(*) AS cnt
04 |     FROM orders o, staffs s
05 |     WHERE o.staff_id = s.staff_id AND o.order_status != 4
06 |     GROUP BY s.staff_id) AS t1
07 | UNION
08 | SELECT id, 0
09 | FROM (
10 |     SELECT DISTINCT s.staff_id AS id
11 |     FROM staffs s
12 |     WHERE s.staff_id NOT IN (
13 |         SELECT DISTINCT s.staff_id AS id
14 |         FROM orders o, staffs s
15 |         WHERE o.staff_id = s.staff_id AND
16 |             o.order_status != 4)
17 |     ) AS t2
18 | ORDER BY cnt DESC, id

```

Let us first determine what each of the subqueries does. Let us start with the following one

```

01 | SELECT s.staff_id AS id, COUNT(*) AS cnt
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND o.order_status != 4
04 | GROUP BY s.staff_id

```

Let us first look at the given SQL database declaration to figure out what the meaning of the predicate `o.order_status != 4` where `o` is an alias for the `orders` table. We find the following comment in the code

```

01 | -- Order status: 1 = Pending; 2 = Processing; 3 = Rejected; 4 = Completed

```

Hence we deduce that `o.order_status != 4` ensures that the orders we select have not yet been completed. We hence select orders which have not yet been completed and match these orders to member of staff via comparing the `staff_id` on the order with the `staff_id` of members of staff. Note that the aliasing are made without the usage of `AS` which is also possible. The following table illustrates the three orders (and the matching staff members) which have not yet been completed.

order_id	order_status	store_id	staff_id	name
3	3	2	2	Mireya Copeland
5	2	4	5	Marcelene Boyer
8	1	2	3	Genna Serrano

We then group the results by `staff_id`. And count the number of uncompleted orders for each `staff_id` which has such uncompleted orders. The results are then returned in a table with one column named `id` containing the `staff_id` and another column containing the number of uncompleted orders for this member of staff. This yields the following table.

id	cnt
2	1
5	1
3	1

Now that we understand this subquery let us move to its outer query.

```

01 | SELECT id, cnt
02 | FROM (
03 |     SELECT s.staff_id AS id, COUNT(*) AS cnt
04 |     FROM orders o, staffs s
05 |     WHERE o.staff_id = s.staff_id AND o.order_status != 4
06 |     GROUP BY s.staff_id) AS t1

```

We do this since the `UNION` splits the query into two subqueries which we can look at separately. We can see that this does not change the result besides aliasing it, as we are selecting all columns of the result and are not changing it in any other way.

We can now start with the second query.

```

01 | SELECT id, 0
02 | FROM (
03 |     SELECT DISTINCT s.staff_id AS id
04 |     FROM staffs s
05 |     WHERE s.staff_id NOT IN (
06 |         SELECT DISTINCT s.staff_id AS id
07 |         FROM orders o, staffs s
08 |         WHERE o.staff_id = s.staff_id AND
09 |             o.order_status != 4)
10 |     ) AS t2

```

We again start with the inner most query.

```

01 | SELECT DISTINCT s.staff_id AS id
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND o.order_status != 4)

```

We can see that this selects distinct `staff_ids` which have uncompleted orders. We then look at query using its result.

```

01 | SELECT DISTINCT s.staff_id AS id
02 | FROM staffs s
03 | WHERE s.staff_id NOT IN (
04 |     SELECT DISTINCT s.staff_id AS id
05 |     FROM orders o, staffs s
06 |     WHERE o.staff_id = s.staff_id AND
07 |         o.order_status != 4)
08 | ) AS t2

```

One can see that we now also select distinct `staff_ids` but remove any that are in the result of the previous query (`NOT IN` does that), i.e. we only select `staff_ids` which have only completed orders. We then also alias this query as `t2`. We can see that this yields the following `staff_ids`.

id
4
1

We can see that this would not work with the `UNION` operator as the first query operand is a table with two rows. We hence use the outer query to add a column with only zeroes to the table. Seeing as the first query comes with the number of uncompleted orders for each `staff_id` this makes a lot of sense, as we now will have a zero in that column for each member of staff which has no uncompleted orders. The result of this query will thus be the following table.

id	cnt
2	1
3	1
5	1
1	0
4	0

We thus conclude that the query list for each member of staff the number of uncompleted orders they have where staff members with more uncompleted orders appear above members with less uncompleted orders.

Query 2: We are given the following SQL query.

```
01 | SELECT oi.product_id, COUNT(DISTINCT customer_id)
02 | FROM order_items oi, products p, orders o
03 | WHERE oi.product_id = p.product_id AND
04 |       oi.order_id = o.order_id AND
05 |       o.order_status = 4
06 | GROUP BY oi.product_id
07 | ORDER BY SUM(quantity * list_price) DESC
```

Let us first notice that we are working with the `order_items` table which means that now we are interested not just in the order itself but also in the items for each order. The `WHERE` clause is used to combine the information in the tables based on if they correspond to each other and in addition to that we have the predicate `o.order_status = 4` which tells us that we only consider completed orders.

We then group the data by `product_id` telling us that we are interested in a metric for each product type. In the select statement we put in one column the `product_id` (which makes sense as we have already concluded that we are interested in a metric for it) and in the other we put the number of distinct customers (identified by their `customer_id`) that ordered this product and have received it (the order has been completed). At last all that remains is to understand what the following statement does.

```
01 | ORDER BY SUM(quantity * list_price) DESC
```

It is important that we use the **SUM** function as otherwise the SQL query would not work. Using the **SUM** function means that we sum `quantity * list_price` over the group which is well-defined. We hence order the result in descending (**DESC**) order according to how much money was spent on the product. The result looks as follows.

product_id	count
14	3
15	2
4	2
7	1
9	1
8	1
1	2
13	1

We hence get a table which shows for each product, which has been ordered at least once how many times it has been ordered and the table is ordered in descending order according by the total money that has been spent on it.

Query 3: We are given the following SQL query.

```
01 | SELECT s.staff_id AS id, COUNT(*) AS cnt
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND
04 |       s.staff_id NOT IN (
05 |         SELECT DISTINCT s.staff_id AS id
06 |         FROM orders o, staffs s
07 |         WHERE o.staff_id = s.staff_id AND
08 |               o.order_status != 4)
09 | GROUP BY s.staff_id
10 | HAVING COUNT(*) > 1
```

We again start with the subquery before considering the entire query.

```
01 | SELECT DISTINCT s.staff_id AS id
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND o.order_status != 4)
```

One can follow from previous reasoning that this yields all distinct staff members (their `staff_ids`) which have uncompleted orders.

Moving to the outer query we then again consider all orders matched to the corresponding staff member, we however remove all rows that are in the result of the subquery and are hence left with staff members without any uncompleted orders. We then group these orders by `staff_id` (one member of staff can handle multiple orders) and then remove any groups for which the staff members has no or only one order (more than one completed and no uncompleted orders). The result then contains one column for the `staff_id` and one column that counts the number of completed orders. The result looks as follows.

id	cnt
4	2
1	3

The query thus selects all members of staff which have more than one completed order and no uncompleted orders.

Query 4: We are given the following SQL query.

```

01 | SELECT S2.product_id, COUNT(*)
02 | FROM (
03 |     SELECT S1.customer_id, S1.product_id, COUNT(*) AS count
04 |     FROM (
05 |         SELECT o.customer_id , o.order_id, oi.product_id
06 |         FROM order_items oi, orders o
07 |         WHERE oi.order_id = o.order_id
08 |         GROUP BY o.customer_id, oi.product_id, o.order_id
09 |     ) AS S1
10 |     GROUP BY S1.customer_id, S1.product_id
11 | ) AS S2
12 | WHERE S2.count > 1
13 | GROUP BY S2.product_id

```

The inner most subquery is given by

```

01 | SELECT o.customer_id , o.order_id, oi.product_id
02 | FROM order_items oi, orders o
03 | WHERE oi.order_id = o.order_id
04 | GROUP BY o.customer_id, oi.product_id, o.order_id

```

We are interested in the ordered items seeing as we select from the `order_items` table. We then match orders and items to be able to combine the information for a given item having been ordered. Let us now look at the `GROUP BY` statement.

```

01 | GROUP BY o.customer_id, oi.product_id, o.order_id

```

The attributes `order_id` and `product_id` uniquely identify the item from all ordered items, as it tells us which item from which order we are considering. Adding the `customer_id` does not change this grouping, it however allows us to add the `customer_id` column to the table, as otherwise this would not be possible. This query hence returns a table of all ordered items and the corresponding customer that put in the order.

In a next step let us consider the outer query which contains this query as a subquery.

```

01 | SELECT S1.customer_id, S1.product_id, COUNT(*) AS count
02 | FROM (
03 |     SELECT o.customer_id , o.order_id, oi.product_id
04 |     FROM order_items oi, orders o
05 |     WHERE oi.order_id = o.order_id
06 |     GROUP BY o.customer_id, oi.product_id, o.order_id
07 | ) AS S1
08 | GROUP BY S1.customer_id, S1.product_id

```

This query now groups the result of the subquery according to the `customer_id` and the `product_id` and hence we get groups which for each customer contains rows of the same product, i.e. we can retrieve information on the number of products a customer ordered for each of the products. The result then adds a column where the number of items per product for each customer is shown.

From here we first remove all products which have not been ordered more than once, we then group the result by `product_id` and display it as a two column table with one column showing the `product_id` and the other column showing the number of times it has been ordered. The result is an empty table as no product exists that has been ordered multiple times in separate ordered items by the same customer, this is surely because the `quantity` attribute is used for whenever items are ordered multiple times in the same order and a non-empty table would suggest some sort of unintended behaviour.

The query select all products which have been ordered multiple times in the same order as separate ordered items by the same customer.

Query 5: We are given the following SQL query.

```

01 | SELECT s.staff_id AS id, COUNT(*) AS cnt
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND
04 |       o.order_status = 4 AND
05 |       s.staff_id IN (
06 |         SELECT s.staff_id AS id
07 |         FROM orders o, staffs s
08 |         WHERE o.staff_id = s.staff_id AND s.staff_id NOT IN (
09 |           SELECT DISTINCT s.staff_id AS id
10 |           FROM orders o, staffs s
11 |           WHERE o.staff_id = s.staff_id AND
12 |                 o.order_status != 4)
13 |         GROUP BY s.staff_id
14 |         HAVING COUNT(*) > 1
15 |       )
16 | GROUP BY s.staff_id

```

We again first consider the inner most query given by

```

01 | SELECT DISTINCT s.staff_id AS id
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND o.order_status != 4)

```

This query selects distinct staff members which have uncompleted orders. Next let us look at the outer query that contains this query as a subquery.

```

01 | SELECT s.staff_id AS id
02 | FROM orders o, staffs s
03 | WHERE o.staff_id = s.staff_id AND s.staff_id NOT IN (
04 |   SELECT DISTINCT s.staff_id AS id
05 |   FROM orders o, staffs s
06 |   WHERE o.staff_id = s.staff_id AND o.order_status != 4)
07 | GROUP BY s.staff_id

```

```
08 | HAVING COUNT(*) > 1
```

We select here all staff members matched to their orders which have no uncompleted orders, because we select all staff members which are not in the result of the sub-query, and then group the result according to the `staff_id`. This assures that we have one entry per staff member in the table. We then count the number of rows per group and remove any entries which have no more than one entry.

This query hence select all staff members that have no uncompleted orders and at least two completed orders.

Query 6: We are given the following SQL query.

```
01 | SELECT id, cnt
02 | FROM (
03 |     SELECT s.staff_id AS id, COUNT(*) AS cnt
04 |     FROM orders o, staffs s
05 |     WHERE o.staff_id = s.staff_id AND
06 |           o.order_status != 4
07 |     GROUP BY s.staff_id) AS t1
08 | ORDER BY cnt DESC, id
```

We select any uncompleted order matched to the corresponding member of staff and group these orders for each member of staff, while counting how many such uncompleted orders we have per member of staff (using the grouping) and then display this in the result table ordered is descending (`DESC`) order according to the number of uncompleted order per staff member and in ascending order by the `staff_id`. This means that the ascending order according to the `staff_id` is only used as a tie breaker (i.e. lexicographic ordering).

Query 7: We are given the following SQL query.

```
01 | SELECT P.product_id, COUNT(DISTINCT customer_id)
02 | FROM (
03 |     SELECT o.customer_id, oi.order_id
04 |     FROM order_items oi, orders o
05 |     WHERE oi.order_id = o.order_id AND o.order_status = 4
06 | ) AS O,
07 | (
08 |     SELECT oi.order_id, p.product_id, p.list_price * oi.quantity AS item_price
09 |     FROM products p, order_items oi
10 |     WHERE oi.product_id = p.product_id
11 | ) AS P
12 | WHERE O.order_id = P.order_id
13 | GROUP BY P.product_id
14 | ORDER BY SUM(item_price) DESC
```

Let us consider first the queries we select the columns from. The first one is given by

```
01 | SELECT o.customer_id, oi.order_id
02 | FROM order_items oi, orders o
03 | WHERE oi.order_id = o.order_id AND o.order_status = 4
```

This query selects all orders matched to the customer which have been completed. The second such query is given by

```
01 | SELECT oi.order_id, p.product_id, p.list_price * oi.quantity AS item_price
02 | FROM products p, order_items oi
03 | WHERE oi.product_id = p.product_id
```

This query results in a table that contains for each ordered item of which **order** it is part of (using the **order_id**), what the product is (using the **product_id**) and the cost of the item which is the product of the item price and the quantity it was ordered.

We now must be careful. We are taking rows from three different tables (**orders**, **order_items** and **products**) hence to avoid duplicates we must make sure that we match all three tables against each other. The following predicate is used to match the subquery **O** with the subquery **P**.

```
01 | WHERE O.order_id = P.order_id
```

This does not suffice to avoid duplicates, as we will account for some items in table **O** and in table **P** as well. Hence we will match the same item in a completed order (in **O**) with all other items inside the order with matching **order_id**. The following result table shows what rows will be accounted for in the result.

customer_id	order_id	product_id	item_price
1	1	15	5999
1	1	15	5999
1	1	13	538
1	1	13	538
2	2	1	379
4	4	4	5798
1	6	9	2999
6	7	8	1799
6	7	8	1799
6	7	8	1799
6	7	1	379
6	7	1	379
6	7	1	379
6	7	4	5798
6	7	4	5798
6	7	4	5798
1	9	14	6998
7	10	14	3499
8	11	7	3999
8	11	7	3999
8	11	7	3999

One can see that for example that the order with **order_id** 7 contains three items. All three items in **O** are matched against all three items in **P**. This query hence produces a rather difficult to explain result.

Query 8: We are given the following SQL query.

```

01 | SELECT product_id, COUNT(DISTINCT customer_id)
02 | FROM (
03 |     SELECT o.customer_id, oi.order_id, oi.item_id
04 |     FROM order_items oi, orders o
05 |     WHERE oi.order_id = o.order_id AND o.order_status = 4
06 | ) AS O,
07 | (
08 |     SELECT oi.order_id, oi.item_id, p.product_id, p.list_price * oi.quantity AS
09 |         item_price
10 |     FROM products p, order_items oi
11 |     WHERE oi.product_id = p.product_id
12 | ) AS P
13 | WHERE O.order_id = P.order_id AND O.item_id = P.item_id
14 | GROUP BY product_id
15 | ORDER BY SUM(item_price) DESC

```

The subqueries aliased with O and P are the same as in query 7. Here we however use the following predicate to match the result of the subqueries

```

01 | WHERE O.order_id = P.order_id AND O.item_id = P.item_id

```

This is how one is supposed to do it as this does not include duplicates. We also notice that we use `product_ip` instead of `P.product_ip`. This however does not make a difference as it will only be included in the columns of subquery P and hence it is clear what column `product_ip` is being referred to.

This query hence sums up all money spend on a product (including it being ordered multiple times in the same order via the `quantity` attribute) and orders the `product_ips` according to it in descending (DESC) order. Alongside the `product_ip` we also find the number of distinct customers which put in orders containing the product.

Exercise 1: We are given the following query description and are tasked to determine which queries fit the description and which do not.

- Returns the product IDs and the total number of customers that have ordered that product.
- The result is ordered in descending order by the total revenue from each product.
- Only completed orders are considered (i.e., `order_status` is 4).

We have looked at each query and what it does. Only query 2, 7 and 8 fit the description, but we have seen that query 7 does not work correctly. Hence we conclude that only query 2 and 8 fit the description.

Exercise 2: We are given the following query description and are tasked to determine which queries fit the description and which do not.

- Return all members of the staff (they need to have an entry in the table `staffs`) by their `staff_id` with their respective number of uncompleted (any other `order_status` than complete) orders.
- If a staff member has no uncompleted order, the corresponding member should be included in the resulting set of the query with '0' as the number of uncompleted orders.

This is exactly what query 1 does all others do not fit the description.

Exercise 3: We are given the following query description and are tasked to determine which queries fit the description and which do not.

- Return only members of the staff (they need to have an entry in the table `staffs`) that do not have any uncompleted but more than one completed order.
- The result consists of the staff ID along with the total number of completed orders.

Without even looking at any other queries we already know that queries 1, 2, 7 and 8 do not fit the description. Query 3 does exactly what the description says, Query 4 does something different, Query 5 fits the description. Hence both queries 3 and 5 fit the description.

Exercise 4: We are given the following query description and are tasked to determine which queries fit the description and which do not.

- Return product IDs and the number of customers who have ordered that product in multiple orders (i.e., strictly more than once).
- Does not include products for which no such customer exists.

That is exactly what Query 4 does, all other queries do not fit the description.

6 ER Model

6.1 HS21, Question 6

Let us quickly recap the most important parts of the Entity-Relationship (ER) model. In the question we are given a ER-model and are tasked to answer questions based on it. Let us look at all distinct features of this model.

- **Entity Set:** The Entity Set is a set of familiar entities in the real world that share the same attributes. Below one can see how an entity set is drawn in the ER model.



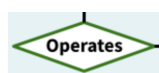
- **Attribute:** An attribute is a property of an object that allows us to distinguish it from other objects. An entity set can have multiple attributes. Below one can see how an attribute is drawn in the ER model, an attribute is connected via an edge to its entity set.



- **Key attribute:** One or more attributes for each entity set are underlined, they are the key attributes and together they compose the **primary key**. Below a key attribute is shown. Any key (a minimal set of attributes whose values identify an entity in the entity set) is called a **candidate key** but only one primary key exists for each entity set. Below one can see how a key attribute may look like in the ER model.



- **Relationship:** Entity sets can engage in relationships with each other. Below one can see how we draw such a relationship in the ER model. We connect



entity sets to relationships using edges. More than two entity sets can engage in a relationship. An entity can even engage in a relationship with itself where this then requires so called **roles** in the relationship (Parent / Child) which we write on the edges.

- **1-to-many relationship:** An entity set A is in a 1-to-many relationship with an entity set B if one A entity can have relationship with **many** B entities, but one B entity can have relationship with **only one** A entity. Below one can find an example of how this looks like in the ER model.



- **many-to-many relationship:** An entity set A is in a many-to-many relationship with an entity set B if one A entity can have relationship with **many** B entities and one B entity can have relationship with many A entities. Below one can find an example of how this looks like in the ER model.



- **1-to-1 relationship:** An entity set A is in a 1-to-1 relationship with an entity set B if an entity A can have relationship with **only one** entity B and an entity B can have relationship **only one** entity A .
- **min-max relationship:** We can also use (min, max) on the edge of (only) binary relationships to restrict the other involved entities number of relationships it can engage in. A 1-to-many relationship where A engages in a 1-to-many relationship with B we would put a $(0, 1)$ on the edge from A (restricting a B entity to only be able to engage with at most one entity from A) and put a $(0, \infty)$ on the edge from B to restrict A not at all.
- **Weak entity set:** A weak entity set relies on an other entity set to exist. We draw weak entities as follows.



- **Weak relationship:** With each weak entity comes a weak relationship to a entity set which its existence depends on. An example for this is shown below.



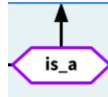
Weak relationships are mostly 1-to-many (the weak entity having the N edge) as a weak entity depends on the entities it engages in weak relationships in and hence cannot be in a weak relationship with multiple entities from the **same** entity set. It however can engage in multiple weak relationships.

- **Weak key attribute:** A weak entity set also needs a primary key, it however always is composed of the primary key alongside some weak key attribute of the weak entity itself (if needed). We underline these attributes with a dashed line as seen below.



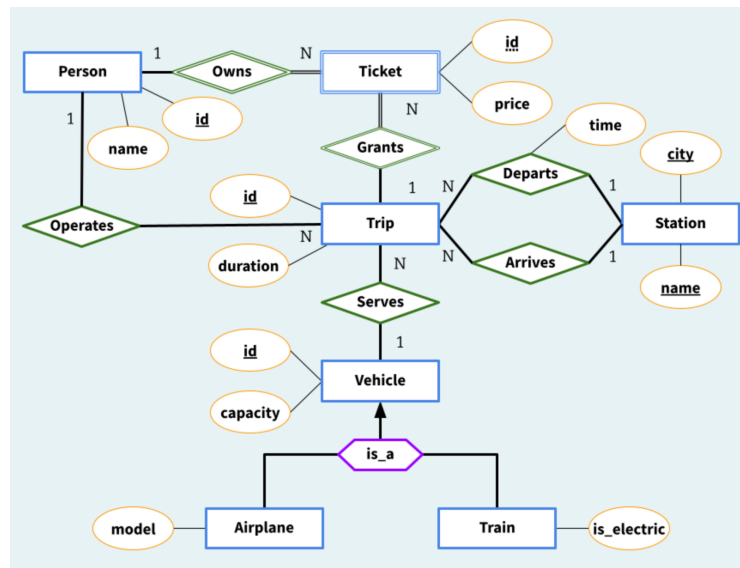
- **Generalisation:** Generalisation is like inheritance in the ER model. It introduces a "is-a" relationships. This means that the entities share the generalized

attributes and thus also the primary key. Hence no attributes of a entity set that is part of a "is-a" relationship (besides the entity set with the primary key as its attributes) has itself key-attributes.



This should cover all important parts of the ER model.

Exercise 1: We are told that all attributes assume global names of the format entityname_attrname and every entity has at most one key. The following ER-model is given to us.



We are asked the following questions.

- A Vehicle can operate multiple Trips at the same time.
 - **TRUE:** The Vehicle entity set engages in a 1-to-many relationship with the Trip entity set, hence a vehicle entity can be in relationship with many trip entities but a trip entity can be in relationship only with one vehicle entity. Hence this statement is correct.
- All Persons pay a different price for the same Trip.
 - **FALSE:** The Ticket entity set is a weak entity that depends on the person entity set hence a Ticket entity can only be in relationship with one Person entity. This however does not mean that the prices must be different, because the price attribute is not part of the key and could thus be the same for any Person entity without being a problem.
- A Ticket is uniquely identified by its ID.

- **FALSE:** A Ticket entities primary key is composed of the primary key of the Person entity combined with its weak key attributes (it is a weak entity), hence a Ticket is not uniquely identified by its id, it requires both the ticket id and the person id to uniquely identify a Ticket entity.
- A Person can own multiple tickets for the same Trip.
 - **TRUE:** We consider the weak relationships the Ticket entity set engages in and see that any Ticket entity can only be in relationship with one Person entity and one Trip entity, this however does not mean that we could have multiple Ticket entities that engage in the same relationships and thus would allow for multiple tickets for the same Trip. This makes sense as each individual Ticket entity still only engages in a relationship with exactly one Person entity and one Trip entity.
- A Ticket must be issued to at least one Passenger.
 - **TRUE:** Ticket entities are weak entities and their existence depends on the existence of a Person entity and the existence of a Trip entity hence if they exist then they stand in relationship with one Person entity and one Trip entity making the Person entity a passenger.
- A Trip can arrive at the same Station as it departed.
 - **TRUE:** Every Trip entity engages in a departs relationship with at most one station entity and a arrives relationships with at most once station entity. We do not restrict if these must be distinct, hence the entities being the same is possible.
- A Trip is served by exactly on Vehicle, which has to be either an Airplane or a Train.
 - **FALSE:** We skip over the first part, as a Vehicle entity can exist without being either an Airplane or a Train entity, hence this statement is wrong.
- A Person is either an Operator of a Trip or a Passenger, which owns a ticket for a trip.
 - **FALSE:** A Person does not have to engage in any relationships, hence it could be neither an Operator nor Passenger.
- There can be multiple Stations with the same name.
 - **TRUE:** The primary key of the Station entity set is composed of both the city and the name of the station, hence if in two distinct cities we have stations with the same name, then that poses no problem to the uniqueness of the primary key and thus this is possible.
- Two trips can depart at the same station at the same time.
 - **TRUE:** The time is not a key attribute hence it does not play a role in determining uniqueness, we can hence have two distinct trips depart at the same station at the same time.

Exercise 2: To translate from an ER model to a relational model we follow the following principles during conversion.

- Entities become relations.
- Relationships become relations.
- We merge relations with the same key.
- Entity sets become tables.
- Attributes of the entity set become attributes of the table.
- A relationship becomes a relation directly when no cardinality constraints are given and the keys of all participating entity sets are part of it.
- When cardinality constraints are given this becomes trickier, as a rule of thumb the key attributes of the entities with the 1 on the edge to the relationship are only contained as normal attributes in the relationship, as one can determine their value from the other attributes (1-to-many relationship). In a N-1-1 relationship we must include one of the key attributes of a relationship with 1 on the edge as key but not both (in addition to the keys of the relationship with an N on the edge).
- Generalization can either be represented by having all attributes and key attributes of the generalising entity set (Vehicle in this ER model) and the attributes of the generalised entity set (Train for example) or by or just the key attributes of the generalising entity set and the attributes of the generalised entity set.
- Roles are turned into key attributes.

We are now given relations and must decide whether or not they are part of the result of translating the ER model from before to a relational model. For the full relational model refer to the solution.

- **Train**(vehicle_id, train_is_electric, vehicle_capacity)
 - **TRUE:** We have the same primary key and all attributes of a train in the ER model are also attributes in the relational model.
- **Departs**(trip_id, departs_time, station_city, station_name)
 - **FALSE:** The Trip entity set is in a many-to-one with the Station entity set relationship (or equivalently the Station entity set is in a one-to-many relationship with the Trip entity set). Because of that the trip_id key suffices to determine the station_city and station_name and hence they cannot be part of the key. This is why including them is incorrect. Keys must always be minimum sets of attributes.
- **Is_A**(vehicle_id, train_id, airplane_id)

- **FALSE:** Generalisation relationships are not being represented as relations only the entities being part of it do.
- **Operates**(trip_id, person_id)
 - The operates relationship is a one-to-many relationship and can thus not contain the key attributes of both entity sets involved in the relationship as its keys. The trip_id suffices to identify the person_id.
- **Ticket**(ticket_id, person_id, trip_id, ticket_price)
 - **FALSE:** One can see that the Ticket entity set is a weak entity set that depends both on the Person entity set and the Trip entity set. We can also see this by it being involved in two many-to-one relationships. The attribute person_id must hence be a key as well as otherwise we cannot uniquely determine the Ticket entity.
- **Station**(station_city, station_name)
 - **TRUE:** This is correct, as the relation includes all keys and attributes of the entity set.
- **Trip**(trip_id, vehicle_id, person_id)
 - **FALSE:** We can certainly merge the Trip relation with the Departs and Arrives relations as they are uniquely determined by the trip_id due to the nature of the one-to-many relationships. We hence would expect the departs_time attribute also to be part of the Trip relation (and others, but this suffices to argue over correctness).
- **Person**(person_id, person_name)
 - **TRUE:** The Person relation contains all attributes and key attributes of the Person Entity set. It is easier to argue for entity sets than for relationships in this regard.
- **Vehicle**(vehicle_id, capacity)
 - **TRUE:** The Vehicle relation contains all attributes and key attributes of the Vehicle Entity set.
- **Airplane**(**vehicle_id**, airplane_model)
 - **TRUE:** As seen we have the choice to also include the other attributes of the Vehicle Entity set as well, but this is also correct.

7 Integrity Constraints

HS21, Question 7

Let us start of with a quick recapitulation of the most important concepts with integrity constraints. The following **constraints** exist.

- **NOT NULL:** The **NOT NULL** constraint is used to force that some attributes of a table cannot be **NULL**, i.e. they must always be set to some value when a new row is inserted into the table.

```
01 | CREATE TABLE person (  
02 |     name VARCHAR(50) NOT NULL  
03 |     age INT);
```

- **PRIMARY KEY:** The **PRIMARY KEY** constraint allows us to force attributes to be unique and not null. It is important that the uniqueness is concerning the whole key, i.e. if a primary key consists of two attributes there can be many keys that share one attribute, they just cannot share both. There can only be one primary key per relation. When a single attribute is the primary key then this can be declared as follows.

```
01 | CREATE TABLE stores (  
02 |     store_id INT PRIMARY KEY,  
03 |     store_name VARCHAR (255) NOT NULL,  
04 | );
```

However we can only declare one attribute as a primary key using this method. Otherwise we declare the primary key afterwards using the following syntax

```
01 | CREATE TABLE order_items(  
02 |     order_id INT NOT NULL,  
03 |     item_id INT NOT NULL,  
04 |     product_id INT NOT NULL,  
05 |     quantity INT NOT NULL,  
06 |     PRIMARY KEY (order_id, item_id)  
07 | );
```

- **UNIQUE:** We can use the **UNIQUE** constraint to force attributes to be unique. We can use the same two methods as above, can however declare multiple attributes as **UNIQUE** without having to use the second method as multiple independent unique attributes can exist without problem.

```
01 | CREATE TABLE students (  
02 |     sID INT UNIQUE,  
03 |     sName VARCHAR (255),  
04 |     GPA REAL,  
05 |     UNIQUE (sID, sName),  
06 |     UNIQUE (sID, GPA));
```

- **CHECK:** The **CHECK** constraint can be used to conduct local value checking of attributes. Check constraints only reject false tuples, unknowns and true are accepted.

```
01 | CREATE TABLE students (  
02 |     sID INT UNIQUE,  
03 |     sName VARCHAR (255),  
04 |     semester INT,  
05 |     GPA REAL,  
06 |     CHECK(GPA + semester = 10));
```

Next we need to look at **referential constraints**. Referential constraints are used to refer to rows from a different table. The attribute can then only take a value that exists in the referenced table or it is **NULL**.

```
01 | CREATE TABLE student_assistant (  
02 |     sID INT REFERENCES students,  
03 |     sName VARCHAR (255) REFERENCES students,  
04 |     lecture VARCHAR (255),  
05 |     workours INT);
```

Referential constraints introduce new problems. What do we do when a row is deleted that is referenced in some other table? Referenced values have to exist as we have seen before or be **NULL**. The following **referential constraints** exist to work against this problem. We can use them in combination with **ON UPDATE** and **ON DELETE** to specify what action should be taken depending on if the referenced value was updated or deleted.

- **CASCADE:** Propagate any updates to tables referencing the row and delete rows that reference rows that are being deleted.
- **RESTRICT:** Prevent deletion of the primary key before trying to do the change and then cause an error.
- **NO ACTION:** Attempt the change then prevent modifications and cause an error.
- **SET DEFAULT:** Sets the references value to the **DEFAULT** value.
- **SET NULL:** Sets the references value to the **NULL**.

RESTRICT and **NO ACTION** are different only based on the ordering (**RESTRICT** throws error immediately while **NO ACTION** throws an error after trying) in which things happen the outcome is the same and in most databases this is not even noticeable. We can now start answering the exercises.

A referenced value is a primary key, or at least unique. We call the attributes that refer to the values **foreign keys**. A primary key can itself never reference a value. The following example shows a table where a **FOREIGN KEY** is declared specifically (MySQL).

```
01 | CREATE TABLE Orders (  
02 |     OrderID int NOT NULL,  
03 |     OrderNumber int NOT NULL,  
04 |     PersonID int,  
05 |     PRIMARY KEY (OrderID),  
06 |     FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
07 | );
```

Exercise 1: We are given three statements and are tasked with determining whether or not they are true.

- The PRIMARY KEY constraint can be augmented with an ON DELETE SET NULL.
 - **FALSE:** A primary key can never reference a row in another table.
- CHECK constraint can be used to reject all inputs.
 - **TRUE:** This is true because we could use a check constraint that always evaluates to false and thus all inputs are rejected.
- UNIQUE constraint does prevent from storing multiple NULL values.
 - The comparison of NULL with any other value (also NULL) using any comparison operator will yield FALSE and hence the UNIQUE constraint cannot prevent us from storing multiple NULL values as they are considered not equal.

Exercise 2: We are given the following schema

```
01 | CREATE TABLE Orders (  
02 |     pid INT,  
03 |     age INT,  
04 |     name TEXT NOT NULL,  
05 |     PRIMARY KEY (pid, age)  
06 | );
```

Initially the table looks as follows.

pid	age	name
1	20	N

We can see that (pid, age) is a primary key, hence the two values cannot be NULL and must be UNIQUE (as a pair, i.e. (1,21) and (1,22) are both valid primary keys).

We are now given two SQL statements and are tasked to determine which of them can be successfully executed.

- The first statement is given by

```
01 | INSERT INTO person VALUES (1, NULL, 'N');
```

- **NO:** This statement cannot be executed successfully as part of the key is NULL which is not possible.

- The second statement is given by

```
01 | INSERT INTO person VALUES (1,21, '');
```

- **YES:** This poses no problem the key contains two non-NULL values and the key is unique in the table as we can distinguish between (1,20) and (1,21).

Exercise 3: We are given the following schema

```
01 | CREATE TABLE Orders (  
02 |     pid INT,  
03 |     age INT,  
04 |     name TEXT NOT NULL,  
05 |     UNIQUE (pid, age)  
06 | );
```

The initial state of the table is given by

pid	age	name
NULL	NULL	'a'

We are again tasked to determine which SQL statements execute successfully.

- The first SQL statement is given by

```
01 | INSERT INTO person VALUES(NULL, NULL, 'M');
```

- **YES:** The **UNIQUE** constraint does not enforce that the value cannot be **NULL** and we have seen that **NULL <> NULL** in SQL (not equal) hence the pair (**NULL**, **NULL**) will always be considered unique no matter what elements are in the table.

- The second SQL statement is given by

```
01 | INSERT INTO person VALUES(20, 20, NULL);
```

- **NO:** The attribute has the **NOT NULL** constraint and we can thus not insert rows into the table for which the **name** attribute is **NULL**.

Exercise 4: We are given the following SQL schemas.

```

01 | CREATE TABLE tab1 ( tab1_key INT PRIMARY KEY );
02 |
03 | CREATE TABLE tab2 (
04 |     tab2_key INT PRIMARY KEY ,
05 |     tab2_ref INT NOT NULL ,
06 |     CONSTRAINT tab2_fk
07 |     FOREIGN KEY ( tab2_ref )
08 |     REFERENCES tab1 ( tab1_key )
09 |     ON DELETE CASCADE
10 |     ON UPDATE CASCADE
11 | );
12 |
13 | CREATE TABLE tab3 (
14 |     tab3_ref INT ,
15 |     CONSTRAINT tab3_fk
16 |     FOREIGN KEY ( tab3_ref )
17 |     REFERENCES tab2 ( tab2_key )
18 |     ON DELETE SET NULL
19 |     ON UPDATE SET NULL
20 | );

```

We can see that the table `tab1` has only one column which is the primary key. We then have the table `tab2` which has one primary key and a foreign key which references `tab1`. It cascades both on delete and update, hence it will update the reference if the original value is updated and delete the row from `tab2` if the value the foreign key references is deleted. Table `tab3` then references the primary key of `tab2` but it sets the reference to `NULL` in both cases (update, delete).

The initial states of the tables are given to us.

tab1_key	tab2_key	tab2_ref	tab3_ref
1	10	1	10
2	20	2	20

We are now tasked with determining how many records each table will contain after executing each of the following statement in order (execution continues even if one statement throws an error).

- The first statement is given by

```

01 | UPDATE tab2 SET tab2_key = 30 WHERE tab2_ref = 1;

```

We can see that this statement attempts to update the `tab2_key` to the value 30 for the row (10, 1). This does not change the number of rows in `tab2`. We now must look on `tab3` as it contains a foreign key referencing `tab2_key`. We see the `ON UPDATE SET NULL` constraint and thus now that the first row in `tab3` is set to `NULL` as it referenced `tab2_key` 10. The table states are now given by

tab1_key	tab2_key	tab2_ref	tab3_ref
1	30	1	NULL
2	20	2	20

- The second statement is given by

```
01 | DELETE FROM tab1 WHERE tab1_key = 2;
```

We now delete the second row from the first table. The table `tab2` contains in the second row a reference to this value (that was deleted) in `tab2_ref`. The foreign key `tab2_ref` cascades on delete and hence we propagate the deletion to `tab2`. Hence the second row of table `tab2` is deleted. Table `tab3` referenced the deleted row in its second row but it has the `ON DELETE SET NULL` constraint and hence sets its entry to `NULL`. This yields the following table states.

tab1_key	tab2_key	tab2_ref	tab3_ref
1	30	1	NULL
			NULL

We have seen that foreign keys can be `NULL`, hence this poses no problem.

- The third statement is given by

```
01 | DELETE FROM tab3 WHERE tab3_ref = 10;
```

This statement does not delete anything as there exists no more row with entry `tab3_ref` equal to 10. Hence the states of the tables remain the same.

8 Functional Dependencies

FS21, Question 8

Let us summarize the most important topics of functional dependencies. Databases can suffer from three main problems

- **Update-Anomaly:** Update anomalies occur when updates to some table in the database is not propagated to other tables where the same information is also stored. Hence modifying data can result in inconsistencies and to errors if there are update anomalies.
- **Insert-Anomaly:** These anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. This can be intentional behaviour of the database when one for example does not specify a primary key properly.
- **Delete-Anomaly:** This type of anomaly occurs when deletion of some record in the database unintentionally deletes other records in the database, and thus caused data loss.

The main cause of these anomalies is **redundancy**. Redundancy allows for a more efficient database but also introduces the anomalies above. Hence we have a trade off between consuming more space (and anomalies) and performance when introducing redundancy. To model this redundancy we use so called **functional dependencies** on a relational model.

Given a schema, i.e. some relation $\mathcal{R}$ and an instance R of this relation, we look on subsets of attributes (subset of columns in an instance of the relation), i.e. $\alpha \subseteq \mathcal{R}$ and $\beta \subseteq \mathcal{R}$ and try to compare α and β . We have the following notions which are important

- **Functional dependency:** We say that there exists a functional dependency $\alpha \rightarrow \beta$ if for any two tuples $r, s \in R$ (R is an instance of the relation $\mathcal{R}$), if r and s share the same values on the columns α , then they also share the same values on the columns β . We write $R \models \alpha \rightarrow \beta$ if R satisfies $\alpha \rightarrow \beta$.
- **Superkey:** A subset of attributes $\alpha \subseteq \mathcal{R}$ if and only if $\alpha \rightarrow \mathcal{R}$ (all attributes of $\mathcal{R}$). A superkey is thus a superset of some other key of the relation $\mathcal{R}$.
- **Minimal functional dependencies:** Given that $\alpha \rightarrow \beta$ if removing any attribute from α does not yield a functional dependency to β anymore, then we say that the functional dependency $\alpha \rightarrow \beta$ is minimal, i.e.

$$\forall A \in \alpha : (\alpha \setminus \{A\} \not\rightarrow \beta)$$

We denote a minimal functional dependency using a dot over the arrow $\alpha \dot{\rightarrow} \beta$.

- **Candidate key:** Any $\alpha \subseteq \mathcal{R}$ is a candidate key if it is a minimal superkey, i.e. if we have the following functional dependency $\alpha \dot{\rightarrow} \mathcal{R}$.

- **Closure:** The closure of set of functional dependencies F (denoted F^+) is given by the set of all functional dependencies implied by F . F **implies** a functional dependency $\alpha \rightarrow \beta$ if every relation instance R of $\mathcal{R}$ that satisfies all functional dependencies in F also satisfies $\alpha \rightarrow \beta$. (Do not bother with this definition for two long if we need to use this there is easier ways to deduce implication.)
- **Equivalent:** Two sets F and G of functional dependencies are called equivalent (denoted $F \equiv G$) if $F \models G$ and $G \models F$, i.e. we can deduce all functional dependencies in G from the ones in F and vice-versa.

The following inference rules are given to us by **Armstrong's Axioms** and these are the ones we are supposed to use when ask to use **Armstrong's Axioms**.

- **Reflexivity:** $\alpha \subseteq \beta \implies \beta \rightarrow \alpha$. This is a so called **trivial** functional dependency.
- **Augmentation:** $\alpha \rightarrow \beta \implies \alpha \cup \gamma \rightarrow \beta \cup \gamma$ where we usually use the notation $\alpha\gamma = \alpha \cup \gamma$. **Transitivity:** $\alpha \rightarrow \beta \wedge \beta \rightarrow \gamma \implies \alpha \rightarrow \gamma$

These axioms are sound and complete, hence any result following by derivation from them is correct and we can derive any correct functional dependency from them. Other rules are given by (very important to know these, remember that $\alpha\gamma := \alpha \cup \gamma$)

- **Union:** $\alpha \rightarrow \beta \wedge \alpha \rightarrow \gamma \implies \alpha \rightarrow \beta\gamma$
- **Decomposition:** $\alpha \rightarrow \beta\gamma \implies \alpha \rightarrow \beta \wedge \alpha \rightarrow \gamma$
- **Pesudo transitivity:** $\alpha \rightarrow \beta \wedge \beta\gamma \rightarrow \theta \implies \alpha\gamma \rightarrow \theta$

In many exercises we are tasked to produce the **closure of α with respect to F** , where F is a set of functional dependencies. We use the notation α^+ and formally define the closure of $\alpha \subseteq \mathcal{R}$ as

$$\alpha^+ = \{y \in \mathcal{R} \mid F \vdash \alpha \rightarrow y\}$$

Computing the closure of a set of attributes α is done by going over all functional dependencies and applying it to the attributes in α , where in each step we add any attributes we could derive using a functional dependency to α until we cannot do this anymore.

There are multiple exercise types that can come up here, the most important once are the following.

- Check if $F \vdash \alpha \rightarrow \gamma$: Here we compute first the closure α^+ and then check whether $\gamma \in \alpha^+$. Of course one can stop when the derivation of the closure includes γ at some point.
- Check if $F \vdash F$: For any functional dependency $\alpha \rightarrow \gamma \in G$ one must verify that we can derive it from F , i.e. we check if $F \vdash \alpha \rightarrow \gamma$. To check whether F and G are equivalent we check $F \vdash G$ and $G \vdash F$.

- Check if $\alpha \subseteq \mathcal{R}$ is a superkey : We verify if from the functional dependencies in F we can derive $\alpha \rightarrow \mathcal{R}$, i.e. we check if $F \vdash \alpha \rightarrow \mathcal{R}$.

Another important topic that appears in this context is the **minimal cover** of a set of functional dependency F . A set G is a minimal cover of F if and only if the following conditions are satisfied

- $G \equiv F$, it must holds that G is equivalent to F .
- All functional dependencies in G have the form $X \rightarrow A$, where A is a single attribute.
- G is minimal, removing any functional dependency from G would make G and F not equivalent anymore. Furthermore removing any attribute from the left hand side of any functional dependency in G , i.e. replacing a functional dependency $XA \rightarrow B$ in G with $X \rightarrow B$, will again make G and F not equivalent anymore.

To compute the minimum basis we first decompose all right hand sides of each functional dependency into a single attribute using the decomposition rule from above. We then remove any trivial functional dependencies, i.e. $\alpha \rightarrow \beta$ where $\alpha \subseteq \beta$. We then attempt to remove attributes from the left hand side of functional dependencies, i.e. if we have for some attribute x that from $X \setminus \{x\} \rightarrow Y$ using F we can derive $X \rightarrow Y \in F$ (without using $X \rightarrow Y$ itself) then we can replace $X \rightarrow Y$ in F with $X \setminus \{x\} \rightarrow Y$ and make the set more minimal. We then again remove any redundant functional dependencies, i.e. ones that occur multiple times or are trivial.

The last topic that comes up with functional dependencies is **decomposition of relations**. The goal of decomposing a relation is to avoid having multiple concepts in one relation and rather have one relation for each concept. We have the following terms that one must know.

- **Lossless:** A decomposition $R \rightsquigarrow R_1, \dots, R_n$ is called lossless if $R = R_1 \bowtie \dots \bowtie R_n$.
- **Preservation:** We say that functional dependencies are preserved if the closure of the functional dependencies $\text{FD}(R)^+$ is equal to the union of the closures of all functional dependencies of the individual relations, i.e.

$$\text{FD}(R)^+ = [\text{FD}(R_1) \cup \dots \cup \text{FD}(R_n)]^+$$

It is important that losslessness does not guarantee preservation of functional dependencies. In many exercises a decomposition of the form $\mathcal{R} = \mathcal{R}_1 \cup \mathcal{R}_2$ is given and this decomposition is lossless if $(\mathcal{R}_1 \cap \mathcal{R}_2) \rightarrow \mathcal{R}_1$ or $(\mathcal{R}_1 \cap \mathcal{R}_2) \rightarrow \mathcal{R}_2$ is satisfied.

A decomposition is given by a projection $\prod_{\mathcal{R}_i}(R) = R_i$ for a given instance R of the relation $\mathcal{R}$.

We are now ready to discuss the exam exercises.

Exercise 1: We are again given the same ER model as in previous questions. We are tasked with determining whether or not given functional dependencies are imposed by the ER model or not.

To determine if a functional dependency is imposed given $\alpha \rightarrow \beta$ we must verify if α is enough to uniquely determine β .

- $\text{trip_id} \rightarrow \text{operates_person_id}$: **TRUE:** Seeing as `Operates` is a one-to-many relationship and that `trip_id` is the entity sets primary key with the N on the edge (two entity sets involved) we can conclude that from this we can determine `person_id`, hence this functional dependency is imposed.
- $\text{ticket_id} \rightarrow \text{trip_id}$: **FALSE:** Now we have a 1-N-N relationship, it however is a weak relationship, hence the primary key of the `Ticket` entity set in the relational model would be `(ticket_id, person_id, trip_id)` as weak entities always have the keys of the entity sets their existence depends on. This functional dependency is hence not imposed by the ER model.
- $\text{station_name} \rightarrow \text{station_city}$: **FALSE:** The primary key of the `Station` relation requires both the `city` and `name` attributes, hence just the `station_name` is not enough to determine the `station_city`, because we have already seen how multiple stations can have the same name but be in different cities. This is hence not imposed by the ER model.
- $\text{ticket_id} \rightarrow \text{ticket_price}$: **FALSE:** The primary key of the `Ticket` relationship is composed of the `ticket_id`, `person_id` and the `trip_id` as discussed above. Hence the `ticket_id` does not suffice to determine uniquely what row in the table we are looking for. Hence this functional dependency is not imposed.
- $\text{person_id} \rightarrow \text{person_name}$: **TRUE:** The `person_id` is the primary key of the `Person` relation and hence suffices to uniquely identify a `Person` entity, and thus a row in a corresponding table. The attribute `person_name` is an attribute of `Person` and hence can be determined using `Person_id`.

Exercise 2: We are given the following relational scheme

$$\mathcal{R}(A, B, C, D, E, F)$$

We are given the following functional dependencies

$$C \rightarrow D \quad B \rightarrow AF \quad DE \rightarrow B$$

This exercise then gives us SQL queries and tasks us to determine whether or not they are always empty. They will structurally as follows.

```
01 | SELECT [LIST OF ATTRIBUTES X]
02 | FROM R
03 | GROUP BY [LIST OF ATTRIBUTES X]
04 | HAVING COUNT(DISTINCT [LIST OF ATTRIBUTES Y]) > 1;
```

What we do now is simply try to derive all attributes in the list Y using the functional dependencies given from the attributes in list X . Let us hence do this.

- The first query is given by

```
01 | SELECT D, E
02 | FROM R
03 | GROUP BY D, E
04 | HAVING COUNT(DISTINCT C) > 1;
```

We can see that the closure of $\{D, E\}$ is given by $\{D, E, B, A, F\}$ (apply $DE \rightarrow B$ first and then $B \rightarrow AF$). C is not in the closure hence we cannot uniquely determine a value for C given only D and E and hence the resulting table might contain more than one row with distinct values in the C column and matching values in the D and E columns. [**Maxbe non-empty**]

- The second query is given by

```
01 | SELECT D, E
02 | FROM R
03 | GROUP BY D, E
04 | HAVING COUNT(DISTINCT F) > 1;
```

We have just seen that F is in the closure of $\{D, E\}$ hence there can be at most one distinct value for F in the result. [**Always empty**]

- The third query is given by

```
01 | SELECT B
02 | FROM R
03 | GROUP BY B
04 | HAVING COUNT(DISTINCT A) > 1;
```

The closure of $\{B\}$ is given by $\{B, A, F\}$ (apply $B \rightarrow AF$) and hence again A is uniquely determined by B and the result can at most contain one distinct result and is thus empty, as we enforce that the number of distinct values for A should be more than one. [**Always empty**]

- The fourth query is given by

```
01 | SELECT C
02 | FROM R
03 | GROUP BY C
04 | HAVING COUNT(D) > 1;
```

Even though D is in the closure of C ($\{C, D\}$) we do not specify if the resulting rows are counted on distinct entries of the D column, hence there could be duplicates and then we would still get a non-empty result. [**Maybe non-empty**]

Exercise 3: We are given a relation $R(A, B, C, D, E)$ with the following dependencies.

$$C \rightarrow A \quad BD \rightarrow E \quad BE \rightarrow AD$$

We are then given sets of attributes and are tasked to determine whether or not they are a candidate key. This can be done by checking if the closure of the given set contains all attributes of the given relation, which tells us that the given set is a super key, we can then verify that it is minimal by removing an attribute and seeing whether or not the closure still contains all attributes of the given relation.

- BCE : We use $C \rightarrow A$ to give us A , then use $BE \rightarrow AD$ and get D and at last $BD \rightarrow E$ to give us E . Hence the closure of $\{B, C, E\}$ is $\{A, B, C, D, E\}$ and we can conclude that this is a super-key, minimality of it then lets us conclude that this is indeed a candidate key. [**Is a candidate key**]
- BCD : From $C \rightarrow A$ we get A and from $BD \rightarrow E$ we get E , hence the closure of $\{B, C, D\}$ is $\{A, B, C, D, E\}$ and we can conclude that this is a super-key, it also is minimal and hence a candidate key. [**Is a candidate key**]
- BE : There is no functional dependency that has C on the right hand side and hence we cannot have it inside of the closure of any set not including C itself. This is thus not a candidate key. [**Is NOT a candidate key**]
- BDE : We again reason as above. [**Is NOT a candidate key**]

9 Normal Forms

FS21, Question 9

We have seen how functional dependencies have come from wanting to formalize the notion of data redundancy. A **normal form** uses functional dependencies to describe the properties of the database design. Given a relation schema R and a set of functional dependencies FD we are interested in the following problems.

- We want to determine whether or not $\{R, FD\}$ satisfies a given normal form.
- If $\{R, FD\}$ satisfies a normal form we want to know what properties this will give us.
- If $\{R, FD\}$ does not satisfy a normal form, we are interested what steps we can take to transform R into R' for which $\{R', FD\}$ does satisfy a normal form.

We will first introduce some concepts and then go through all normal forms discussed in this lecture. Let us first look at **Multi-Value Dependency** (MCD). We define a MVD on a relation $R(\alpha, \beta, \gamma)$ such that if $\alpha \twoheadrightarrow \beta$ then we can decompose R losslessly into

$$R_1 = \Pi_{\alpha, \beta}(R) \quad \text{and} \quad R_2 = \Pi_{\alpha, \gamma}(R)$$

We thus have $R = R_1 \bowtie R_2$. The formal decomposition is more complicated than this, but understanding this should suffice. The following laws hold for multi-value dependencies

- **Trivial MVDs:** For $\mathcal{R}(\alpha, \theta)$ we have $\alpha \twoheadrightarrow \alpha\theta$, i.e. $\alpha \twoheadrightarrow \mathcal{R}$. We also have for $\mathcal{R}(\alpha, \theta)$ that $\alpha \twoheadrightarrow \theta$, i.e. $\alpha \twoheadrightarrow \mathcal{R} \setminus \alpha$. And at last similarly to the functional dependency we have that from $\beta \subseteq \alpha$ it follows that $\alpha \twoheadrightarrow \beta$.
- **Promotion:** $\alpha \rightarrow \beta \implies \alpha \twoheadrightarrow \beta$ (the converse is however not true)
- **Complement:** $\alpha \twoheadrightarrow \beta \implies \alpha \twoheadrightarrow \mathcal{R} \setminus (\alpha \cup \beta)$.
- **Multi-Value Augmentation:** $\alpha \twoheadrightarrow \beta \wedge (\delta \subseteq \gamma) \implies \alpha\gamma \twoheadrightarrow \beta\delta$
- **Multi-Value Transitivity:** $(\alpha \twoheadrightarrow \beta) \wedge (\beta \twoheadrightarrow \gamma) \implies \alpha \twoheadrightarrow \gamma$

Now let us look at the normal forms.

- **1NF:** The first normal form dictates that a database must have only atomic domains, i.e. no arrays, lists, etc. In other words we want basic datatypes to be the elements of our database. This makes the concept of a key well-defined.
- **2NF:** A relation R is in the second normal form if and only if every non-key attribute is minimally dependent on every key, i.e. no non-key attribute depends only on a part of some key. Relations that are not in 2NF suffer from all three types of anomalies. 2NF gets rid of insertion anomalies, the other two types still remain.
- **3NF:** A relation R is in the third normal form if and only if for all functional dependencies at least one of the following conditions hold

- $B \in \alpha$ (it is a trivial FD)
 - B is an attribute of at least one key.
 - α is a superkey of R .
- **BCNF:** A relation R is in Boyce-Codd Normal Form if and only if for all functional dependencies at least one of the following conditions hold
 - $B \in \alpha$ (it is a trivial FD)
 - α is a superkey of R .

The Boyce-Codd normal form assures that in each relation information is stored only once, which avoid data redundancy.

- **4NF:** A relation R is in the fourth normal form if and only if for all $\alpha \twoheadrightarrow \beta$ at least one of the following conditions hold
 - $\alpha \twoheadrightarrow \beta$ is trivial. (Remember that there are multiple ways for a MVD to be trivial)
 - α is a superkey of R .

There is no best normal form. As already mentioned more data redundancy makes a database faster, it however introduces also more anomalies. 3NF does not get rid of insert / delete anomalies, BCNF does get rid of them. The normal forms follow a hierarchy, where any normal form in the above explanation satisfies all conditions of the previous ones, i.e. 3NF is also 2NF and 1NF.

For exercises we are often tasked to apply the **decomposition** and **synthesis** algorithm. Let us discuss what they do. The decomposition takes as input a relational schema $\mathcal{R}$ and outputs a lossless decomposition $\mathcal{R}_1, \dots, \mathcal{R}_2$ such that they will perfectly recover the information in $\mathcal{R}$ and $\mathcal{R}_1, \dots, \mathcal{R}_2$ are in BCNF. It is important that this does not necessarily preserve functional dependencies. The algorithm is given as follows. An evil functional dependency is an FD that violates the condition of the BCNF.

Algorithm 1 Decomposition($\mathcal{R}$)

```

Result  $\leftarrow \{\mathcal{R}\}$ 
while  $\exists R_i$  in Result such that  $R_i$  is not BCNF do
  Let  $\alpha \rightarrow \beta$  be the evil FD
   $\mathcal{R}_i^1 = \alpha \cup \beta$ 
   $\mathcal{R}_i^2 = \mathcal{R}_i - \beta$ 
  Result = (Result  $- \mathcal{R}_i$ )  $\cup \{\mathcal{R}_i^1, \mathcal{R}_i^2\}$ 
end while
return Result
  
```

The synthesis algorithm takes as input a relational schema $\mathcal{R}$ and outputs a lossless decomposition $\mathcal{R}_1, \dots, \mathcal{R}_n$ in 3NF. It preserves all functional dependencies, it however still contains data redundancies caused by functional dependencies. The algorithm is given as follows.

Algorithm 2 Synthesis($\mathcal{R}$)

 Compute the minimal basis F_c of F

 For all $\alpha \rightarrow \beta \in F_c$, create $\mathcal{R}_{\alpha \cup \beta}(\alpha \cup \beta)$

If none of the above relations contains a superkey, add a relation with a key

 Eliminate $\mathcal{R}_\alpha$ if there exists $\mathcal{R}_{\alpha'}$, such that $\alpha \subseteq \alpha'$.

There also exists a decomposition algorithm for 4NF which is almost the same as the one for BCNF.

We thus have found decomposition which preserve functional dependencies up to 3NF and lossless decomposition up to 4NF.

Algorithm 3 Decomposition($\mathcal{R}$)

 Result $\leftarrow \{\mathcal{R}\}$
while $\exists R_i$ in Result such that R_i is not 4NF **do**

 Let $\alpha \twoheadrightarrow \beta$ be the evil FD

 $\mathcal{R}_i^1 = \alpha \cup \beta$

 $\mathcal{R}_i^2 = \mathcal{R}_i - \beta$

 Result = (Result $- \mathcal{R}_i$) $\cup \{\mathcal{R}_i^1, \mathcal{R}_i^2\}$
end while
return Result

Let us now solve the exam exercises.

Exercise 1: We are given the relational scheme $\mathcal{R}(A, B, C, D, E)$ with the following set of functional dependencies.

$$AC \rightarrow B \quad D \rightarrow AB \quad CE \rightarrow BD$$

We are then tasked to determine whether a given set of attributes is a candidate key or not. Remember that a given set of attributes is a candidate key if it is minimal and a superkey (its closure contains all attributes of the relation).

- $\{A, C\}$: E cannot be derived from any of the functional dependencies hence any set not containing E can immediately be ruled out as a potential candidate key. **Is NOT a candidate key**
- $\{D, E\}$: C can also not be derived from the given functional dependencies hence any set not containing C can also immediately be ruled out as a potential candidate key. **Is NOT a candidate key**
- $\{C, D, E\}$: One can see that using the rule $CE \rightarrow BD$ we can derive D hence this set cannot be minimal as removing D from it would not change its closure. **Is NOT a candidate key**
- $\{C, E\}$: We can see that C and E must both be in the candidate key, hence this certainly is minimal if it were a key. We verify that using $CE \rightarrow BD$ we can derive B and D and using $D \rightarrow AB$ we then derive A which means that the closure contains all attributes of the relation. Hence this is a candidate key. **Is a candidate key**

Exercise 2: We are tasked with applying the synthesis algorithm to $\mathcal{R}$ (from the previous exercise) to produce a decomposition in 3NF. We are then given relations and must decide whether or not they are part of the decomposition.

- **Step 1:** Compute the minimum basis. We first decompose the right hand sides of the functional dependencies which yields

$$AC \rightarrow B \quad D \rightarrow A \quad D \rightarrow B \quad CE \rightarrow B \quad CE \rightarrow D$$

There are no trivial FDs. There also are not redundant attributes. $CE \rightarrow B$ however is redundant. This is because from $CE \rightarrow D$ we can derive D and from $D \rightarrow B$ we can derive B , hence this FD can be removed. We are left with the following set of FDs, which is the minimal basis.

$$AC \rightarrow B \quad D \rightarrow A \quad D \rightarrow B \quad CE \rightarrow D$$

- **Step 2:** For all $\alpha \rightarrow \beta$ we now create the relation $\mathcal{R}_{\alpha \cup \beta}(\alpha \cup \beta)$ this yields the following relations.

$$\mathcal{R}_{abc}(A, B, C) \quad \mathcal{R}_{ad}(A, D) \quad \mathcal{R}_{bd}(B, D) \quad \mathcal{R}_{cde}(C, D, E)$$

- **Step 3:** We check that at least one of the relations contains a super key which $\mathcal{R}_{cde}(C, D, E)$ does.
- **Step 4:** We find no relations $\mathcal{R}_\alpha$ and $\mathcal{R}_{\alpha'}$ for which $\alpha \subseteq \alpha'$.

Solving the exercise is hence straight forward as we must just compare if the given relation is in the result we just computed.

- $\mathcal{R}_{abc}(A, B, C)$: **Is part of result**
- $\mathcal{R}_{ace}(A, C, E)$: **Is NOT part of result**
- $\mathcal{R}_{cde}(C, D, E)$: **Is part of result**
- $\mathcal{R}_{bce}(B, C, E)$: **Is NOT part of result**

Exercise 3: We are given the relational scheme $\mathcal{S}(A, B, C, D, E)$ and the following set of functional dependencies

$$AC \rightarrow EB \quad DCB \rightarrow E \quad CE \rightarrow D \quad D \rightarrow A$$

We are tasked to decompose the relation $\mathcal{S}$ into BCNF using the decomposition algorithm. Let us hence do that.

Even though the algorithm does not mention it we first need to determine the candidate keys for the relation. Otherwise we cannot determine whether or not FDs violate BCNF. We remember that for a functional dependency $\alpha \rightarrow \beta$ it does not violate BCNF (i.e. it is not an evil FD) if and only if either is a trivial FD or if α is a superkey (superset of some candidate keys) of the relation $\mathcal{S}$. Let us hence first compute the candidate keys.

We first note that C must always be part of the candidate key as none of the given FDs can be used to derive C . We can also conclude that there must be at least two elements in any candidate key as the only FD which has a single attribute on the left hand side is $D \rightarrow A$ and $\{A, D\}$ does not have $\mathcal{S}$ as its closure (its closure is $\{A, D\}$). Let us hence look for keys with two attributes. We find that $\{A, C\}$ has the closure $\mathcal{S}$ (apply $AC \rightarrow EB$ to give us B and E and then apply $CE \rightarrow D$ to give us D). Hence $\{A, C\}$ is a candidate key as it is a key and it is minimal. The same goes for $\{C, E\}$ and for $\{C, D\}$. We hence find the following candidate keys

$$\{A, C\} \quad \text{and} \quad \{C, E\} \quad \text{and} \quad \{C, D\}$$

Let us now execute the algorithm.

- **Step 1:** We find no trivial FDs and hence look for FDs $\alpha \rightarrow \beta$ for which α is not a superkey. $D \rightarrow A$ is such an FD and is thus our evil FD. We have Result = $\{\mathcal{S}\}$.
- **Step 2:** We define $\mathcal{S}_i^1 = \{A, D\}$ and $\mathcal{S}_i^1 = \{A, B, C, E\}$.
- **Step 3:** The result is updated to Result = $\{\mathcal{S}_i^1, \mathcal{S}_i^2\} = \{\{A, D\}, \{A, B, C, E\}\}$
- **Step 4:** We repeat the process where for $\{A, D\}$ we only have the functional dependency $A \rightarrow D$ and thus $\{A\}$ is a candidate key. We hence find no more evil FDs and are done.

The result of the decomposition is given by

$$\mathcal{S}_{a,d}(A, D) \quad \mathcal{S}_{a,b,c,e}(A, B, C, E)$$

We can hence answer the given questions from this knowledge.

- $\mathcal{S}_{a,d}(A, D)$: **Is part of the result**
- $\mathcal{S}_{c,d,e}(C, D, E)$: **Is NOT part of the result**
- $\mathcal{S}_{b,c,d,e}(B, C, D, E)$: **Is part of the result**
- $\mathcal{S}_{b,c,e}(B, C, E)$: **Is NOT part of the result**

Exercise 4: We are given the following relational scheme $\mathcal{Q}(A, B, C, D, E)$ with the functional dependency $ABC \rightarrow DE$ and the MVD $AB \twoheadrightarrow CD$. We again first determine the candidate key and find (A, B, C) (keys come from FDs). We then apply the algorithm as before but check for evil MVDs instead. One notices that MVD that contain all attributes of the relation are trivial, this is important for such exercises (is in the definition of trivial MVDs). We find that $AB \twoheadrightarrow CD$ is neither trivial nor is AB a superkey. We hence decompose the relation into

$$\mathcal{Q}_{a,b,c,d}(A, B, C, D) \quad \mathcal{Q}_{a,b,e}(A, B, E)$$

At this point we find no more evil MVDs and are thus done. All that is left is to answer the questions.

- $\mathcal{Q}_{a,b,c,d}(A, B, C, D)$: **Is part of result**
- $\mathcal{Q}_{a,b,c}(A, B, C)$: **Is NOT part of result**
- $\mathcal{Q}_{a,b,d}(A, B, D)$: **Is NOT part of result**
- $\mathcal{Q}_{a,b,e}(A, B, E)$: **Is part of result**

Exercise 5: For the sake of the exercise we will assume that $e' \neq e$ (same for the others). We know that (A, B, C) is a candidate key (there are no other candidate keys - this is important). A table for the relation Q is given to us by

Tuple	A	B	C	D	E
t	a'	b	c	d	e
u	a	b	c'	d'	e'

- $v(a, b, c', d', e)$: Assuming $e \neq e'$ this cannot be in the table as there already is a row with key (a', b, c') and hence this is not in the table. **Can not be in Q**
- $v(a, b, c', d, e)$: Assuming $e \neq e'$ or $d \neq d'$ this cannot be in the table as there already is a row with key (a, b, c') and hence this is not in the table. **Can not be in Q**
- $v(a, b, c, d', e')$: There is now row in the table currently with the key (a, b, c) hence this could be in the table. (Again assuming that $a' \neq a$ or $c \neq c'$) **Might be in Q**
- $v(a, b, c, d, e')$: $v(a, b, c, d', e')$: There is now row in the table currently with the key (a, b, c) hence this could be in the table. (Again assuming that $a' \neq a$ or $c \neq c'$) **Might be in Q**

10 Query Processing

I recommend reading J.Stauffers summary for this part as it is what I used to understand this part and it mostly is learning by heart.

11 Transactions

HS21, Question 11

As seen before two instructions (in this case SQL queries) can be interleaved to produce problems. When a system fails during changes to the database, there are three possible state which the database can be in

- The failure occurred after all changes have been made.
- The failure occurred before any changes have been made.
- The failure occurred such that some changes have been made but not all changes have been made.

The problematic state is the last one as it results in inconsistent data. Transactions are made concurrently and they appear to be made in isolation, because if the system fails each transaction's changes are either completely made or not made at all. A transaction is initiated with the **BEGIN** keyword, terminated with the **COMMIT** keyword (all changes are made) and aborted with the **ABORT** (no changes are made) keyword. This works due to any partial changes being made being able to be reverted by doing a rollback.

A transaction has the following desired properties

- **Atomicity:** A transaction is executed completely or not at all.
- **Consistency:** A fully executed transaction over a consistent DB produces a consistent DB.
- **Isolation:** A transaction executed as if it were the only transaction being executed.
- **Durability:** Committed changes of a transaction are never lost (can be recovered).

We abbreviate this by **ACID**. Let us take a closer look on how to implement isolation. Dirty reads are reads where another uncommitted transaction has written to the read location.

- **Serializable:** An execution of transactions that is equivalent to some sequential order of the transactions is called serializable.
- **Repeatable Read:** No dirty reads are allowed and an item read multiple times cannot change value. This is weaker as serializability.

- **Read committed:** No dirty reads are allowed. It is weaker than repeatable read.
- **Read uncommitted:** Transactions are allowed to perform dirty reads.

We model a transactions as a sequence of reads and writes. We write $a \leftarrow R(A)$ to signify that object A is read into variable a . We write $W(B, b)$ to signify that we write the value of variable b into object B . We assume that the moment we write the changes are reflected in the database. We now look at the notion of **conflict serializability**. We call a schedule conflict serializable if the schedule is conflict equivalent to some serial schedule. Two schedules are conflict equivalent if and only if

- They involve the same actions of the same transactions.
- Every pair of conflicting actions is ordered in the same way.

There are different types of conflicts.

- **Trivial Conflict:** Any two different operations of the same transactions are conflicting, we can hence not reorder within transactions.
- **Read-Write Conflict:** Read after Write can lead to unrepeatable reads as we read before the value is changed and afterwards it will be different. Below one can see an example of this.

01	----- T1 -----	----- T2 -----
02	BEGIN	
03	Read(A)	
04		BEGIN
05		Write(A)
06		COMMIT
07	Read(A)	
08	COMMIT	

- **Write-Read Conflict:** Write before read leads to dirty reads.
- **Write-Write Conflict:** Two writes that are interleaved to the same variable may overwrite uncommitted data.

How do we decide conflict serializability? We construct a dependency graph where each transaction is a node in the graph and there is an edge from T_i to T_j if an operation o_i in T_i is in a conflict with an operation o_j in T_j and o_j appears earlier than o_i . Any schedule for which this dependency graph is acyclic is conflict serializable.

Let us now move on to the exercises.

Exercise 1: We are told that in this (and the following) exercise the following notation is used.

- $r_i(A)$: Transaction i reads data object A .

- $w_i(A)$: Transaction i writes to data object A .
- b_i : Begin of transaction i .
- c_i : Transaction i commits successfully.
- a_i : Transaction i aborts.

Once we determine that a schedule is conflict serializable we attempt to swap non-conflicting operations until we find a serial schedule. We construct the dependency graph and then search for a topological ordering if it exists.

- We are given the following History

$$r_1(A), r_1(B), r_2(A), w_1(A), w_1(B), w_3(B), w_3(A)$$

We find the conflicts $r_2(A) \rightarrow w_1(A)$ and $w_1(B) \rightarrow w_3(B)$ (these are conflict and the one on the left side of the arrow comes before the one on the right side in the history.). We find however no cycles in our dependency graph. From these two dependencies we find that if we were to do T_2 before T_1 the first one cannot happen and T_2 before T_3 the second one cannot happen. Hence our serialization is given by $T_2T_1T_3$.

- We are given the following History

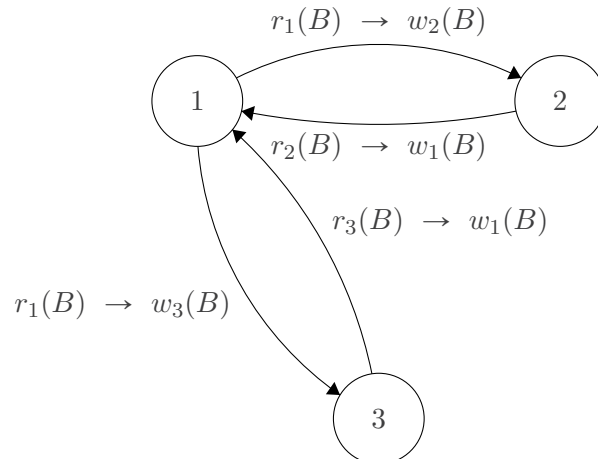
$$r_3(A), r_3(B), r_2(A), w_2(A), r_1(A), w_2(B), w_1(A), w_3(B)$$

We find in our dependency graphs the and edge from node 2 to node 3 and one from node 3 to node 2. This is a cycle and is caused by the dependencies $r_3(B) \rightarrow w_2(B)$ and $r_3(A) \rightarrow w_2(A)$. This history is hence not conflict serializable.

- We are given the following History

$$r_1(B), r_2(A), r_3(B), w_2(A), w_3(B), w_1(B)$$

This time we find multiple cycles in our dependency graph.



The figure above shows just some of the edges and conflict we would also find and edge for conflict between writes, for example the conflict $w_2(B) \rightarrow w_1(B)$. This history is not conflict serializable.

- We are given the following History

$$r_1(B), r_2(A), w_1(B), r_2(A), w_1(A), r_3(A), w_3(B), w_2(B)$$

We find the cycle $r_2(B) \rightarrow w_1(B)$ and $w_1(B) \rightarrow w_2(B)$ (and others) in our dependency graph and hence this history is not conflict serializable.

- We are given the following History

$$r_1(B), r_2(A), r_3(A), r_3(B), w_3(A), w_1(B),$$

We find the dependencies $r_2(A) \rightarrow w_3(A)$ and $r_3(B) \rightarrow w_1(B)$ but no cycles and hence conclude from that that the serialization order must be $T_2T_3T_1$.

- We are given the following History

$$r_1(A), r_2(A), r_2(B), w_1(A), w_2(B)$$

We find the dependency $r_2(A) \rightarrow w_1(A)$ but no cycle and hence conclude that the serialization order must be T_2T_1

Exercise 2: Any conflict serializable schedule is serializable but not vice-versa. Hence there exists schedule which are not CS but serializable. This is important.

- We are given the following History

$$r_1(A), w_2(A), w_1(A), w_3(A)$$

This history is serializable we find the serialization $T_1T_2T_3$. It however is not CS as we find the cycle using $r_1(A) \rightarrow w_2(A)$ and $w_2(A) \rightarrow w_1(A)$.

- We are given the following History

$$r_1(A), w_2(A), r_3(A), w_1(A), w_3(A)$$

Here it is more difficult to determine a serial order as it again is not CS. We want to find a serial order of T_1 , T_2 and T_3 such that is is equivalent to the execution of the history in sequential order. It is apparent that T_1 must come first as $r_1(A)$ is the first read and all other transaction have a write as their first operation. We then see that $r_3(A)$ would read from $w_2(A)$ in sequential order and thus we find that T_2 must come before T_3 . This yields a valid serialization $T_1T_2T_3$.

- We are given the following History

$$r_1(A), r_1(B), w_1(A), r_2(B), w_3(B), w_1(A), r_2(B)$$

We can see that we have a write from a different transaction between two reads from the same transaction. This can never be serializable, as both reads $r_2(B)$ would read the same value in serial ordering which is not the case if we were to execute history 3 in sequential order.

- We are given the following History

$$r_1(B), r_2(A), w_2(A), r_1(A), w_1(A), w_1(B),$$

We find that this is CS and hence also serializable when drawing the dependency graph. Seeing as in serial order $r_1(A)$ reads the result of $w_2(A)$ the serialization must be T_2T_1 .

- We are given the following History

$$r_1(A), r_2(A), w_1(A), w_2(A)$$

It is impossible that $r_1(A)$ and $r_2(A)$ read the same value in any serialization, as the other transaction writes to A as well (assuming it writes a different value), hence this is not serializable.

12 Recovery

FS21, Question 12

We can classify schedules into classes depending on what recoverability properties it has.

- **Recoverable Schedule (RC):** If T_i reads X from T_j and commits, then $c_j < c_i$ (The write of T_j commits before the read of T_i commits) where c_i is the commit time of T_i , then we call the schedule RC (if this is true for any $i \neq j$). We do not need to undo committed transactions here.
- **Avoiding Cascading Abort Schedule (ACA):** If T_i reads X from T_j , then $c_j < r_i[X]$, where $r_i[X]$ is the time T_i reads X . This means that T_i reads a committed value. Aborting a transaction does not cause aborting others.
- **Strict Schedule (ST):** If T_i reads from or overwrites a value X written by T_j then $c_j < r_i[X] \wedge c_j < w_i[x]$ or $a_j < r_i[X] \wedge a_j < w_i[X]$. Hence either T_j commits or aborts the changes before T_i reads / writes. Undoing a transaction does not undo the changes of other transactions.

Exercise 1: We are given histories and are tasked to indicate the strictest recoverability class. We have that Strict is stricter than ACA which is stricter than recoverable which is stricter than not recoverable.

- We are given the following History

$$w_1(A), r_2(A), c_1, c_2$$

This schedule is not ACA because we have a write from T_1 before a read from T_2 but the read happens before the commit of T_1 , hence the condition is violated, i.e. $c_1 \not< r_2[A]$. It is recoverable however as we have $c_1 < c_2$.

- We are given the following History

$$r_1(A), w_2(A), c_2$$

We have no read after writes or writes after writes, hence this schedule is strict.

- We are given the following History

$$w_1(A), w_2(A), w_2(B), r_1(A), c_1, c_2$$

We can see that T_2 writes to A and T_1 reads from A before T_1 commits making this schedule not recoverable.

- We are given the following History

$$w_1(A), w_2(A), c_1, c_2$$

We have no read before writes and hence this schedule is already ACA. We do have an overwrite where T_2 overwrites the value written by T_1 but T_1 does not commit before the write happens and hence this is not strict, i.e. $c_1 \not< w_2[A]$.

- We are given the following History

$$w_1(A), c_1, r_2(A), c_2$$

This is a strict schedule as it is serial which is even stronger than strict.

- We are given the following History

$$w_1(A), a_1, r_2(A), c_2$$

This is also a serial schedule and hence strict.

- We are given the following History

$$w_1(A), r_2(A), c_2, c_1$$

This schedule has a write from T_1 before a read from T_2 but T_1 commits before T_2 does, hence this is not recoverable.

- We are given the following History

$$w_1(A), r_2(A), a_1, a_2$$

Here the issue from before is lifted as we can treat aborts and commits the same for these types of exercises and now we have $a_1 < a_2$ hence it is recoverable. It however is not ACA because $a_1 \nmid_2 [A]$.

- We are given the following History

$$w_1(A), r_1(B), w_2(B), c_1, c_2$$

This schedule has no write before read or write before write to the same object hence this is a strict schedule.

- We are given the following History

$$w_1(A), r_2(A), c_2, a_1$$

T_1 writes to A before T_2 reads from A and T_2 commits before T_1 aborts hence this is not recoverable.

- We are given the following History

$$w_1(A), w_2(B), r_2(A), c_1, a_2$$

Here T_1 writes to A before T_2 reads to A but we have that T_1 commits before T_2 does. It is not ACA as T_2 reads from A before T_1 commits. It is thus recoverable.

13 SQL Code

FS21, Question 3

```
01 | CREATE TABLE [dbo].[CisDuvodNezajmu] (  
02 |     [OID] [ INT ] IDENTITY(1,1) NOT NULL INT,  
03 |     [Nazev] [NVARCHAR](100) COLLATE Czech_CI_AS NULL,  
04 |     [Cre] [DATETIME] NULL,  
05 |     [CreBy] [INT] NULL,  
06 |     [Mod] [DATETIME] NULL,  
07 |     [ModBy] [INT] NULL,  
08 |     [OptimisticLockField] [INT] NULL,  
09 |     [GCRecord] [INT] NULL,  
10 |     CONSTRAINT[PK_CisDuvodNezajmu] PRIMARY KEY CLUSTERED ([OID])  
11 | );  
12 | CREATE TABLE stores (  
13 |     store_id INT NOT NULL,  
14 |     store_name VARCHAR (255) NOT NULL,  
15 |     PRIMARY KEY (store_id)  
16 | );  
17 |  
18 | CREATE TABLE staffs (  
19 |     staff_id INT NOT NULL,  
20 |     name VARCHAR(50) NOT NULL,  
21 |     store_id INT NOT NULL,  
22 |     PRIMARY KEY(staff_id)  
23 | );  
24 |  
25 | CREATE TABLE categories (  
26 |     category_id INT NOT NULL,  
27 |     category_name VARCHAR(255) NOT NULL,  
28 |     PRIMARY KEY (category_id)  
29 | );  
30 |  
31 | CREATE TABLE products (  
32 |     product_id INT NOT NULL,  
33 |     product_name VARCHAR (255) NOT NULL,  
34 |     category_id INT NOT NULL,  
35 |     model_year INT NOT NULL,  
36 |     list_price INT NOT NULL,  
37 |     PRIMARY KEY (product_id)  
38 | );  
39 |  
40 | CREATE TABLE orders (  
41 |     order_id INT NOT NULL,  
42 |     customer_id INT,  
43 |     order_status INT NOT NULL,  
44 |     -- Order status: 1 = Pending; 2 = Processing; 3 = Rejected; 4 = Completed  
45 |     order_date DATE NOT NULL,  
46 |     store_id INT NOT NULL,  
47 |     staff_id INT NOT NULL,  
48 |     PRIMARY KEY (order_id)  
49 | );  
50 |  
51 | CREATE TABLE order_items(  
52 |     order_id INT NOT NULL,  
53 |     item_id INT NOT NULL,
```

```

54 |     product_id INT NOT NULL,
55 |     quantity INT NOT NULL,
56 |     PRIMARY KEY (order_id, item_id)
57 | );
58 |
59 | INSERT INTO stores (store_id, store_name) VALUES
60 | (1, "Santa Cruz Bikes"),
61 | (2, "Baldwin Bikes"),
62 | (3, "Rowlett Bikes"),
63 | (4, "Simply Bikes");
64 |
65 | INSERT INTO staffs (staff_id, name, store_id) VALUES
66 | (1, "Fabiola", 1),
67 | (2, "Mireya Copeland", 2),
68 | (3, "Genna Serrano", 2),
69 | (4, "Virgie Wiggins", 3),
70 | (5, "Marcelene Boyer", 4);
71 |
72 | INSERT INTO categories (category_id, category_name) VALUES
73 | (1, "Comfort Bicycles"),
74 | (2, "Electric Bicycles"),
75 | (3, "Mountain Bicycles"),
76 | (4, "Road Bikes"),
77 | (5, "TT Bikes");
78 |
79 | INSERT INTO products (product_id, product_name, category_id, model_year,
80 |     list_price) VALUES
81 | (1, "Trek 820 - 2016", 3, 2016, 379),
82 | (2, "Ritchey Timberwolf Frameset - 2016", 3, 2016, 749),
83 | (3, "Surly Wednesday Frameset - 2016", 3, 2016, 999),
84 | (4, "Trek Fuel EX 8 29 - 2016", 1, 2016, 2899),
85 | (5, "Heller Shagawaw Frame - 2016", 1, 2016, 1320),
86 | (6, "Surly Ice Cream Truck Frameset - 2016", 3, 2016, 469),
87 | (7, "Trek Slash 8 27.5 - 2016", 3, 2016, 3999),
88 | (8, "Trek Remedy 29 Carbon Frameset - 2016", 3, 2016, 1799),
89 | (9, "Trek Conduit+ - 2016", 2, 2016, 2999),
90 | (10, "Canyon Speedmax CFR - 2016", 5, 2016, 12010),
91 | (11, "Surly Straggler 650b - 2016", 4, 2016, 1680),
92 | (12, "Electra Townie Original 21D - 2016", 2, 2016, 549),
93 | (13, "Electra Cruiser 1 (24-Inch) - 2016", 2, 2016, 269),
94 | (14, "Trek Domane SL 6 - 2017", 4, 2017, 3499),
95 | (15, "Trek Silque SLR 7 Women's - 2017", 4, 2017, 5999);
96 |
97 | INSERT INTO orders (order_id, customer_id, order_status, order_date, store_id,
98 |     staff_id) VALUES
99 | (1, 1, 4, 2016-01-01, 1, 1),
100 | (2, 2, 4, 2016-01-01, 2, 3),
101 | (3, 3, 3, 2016-01-02, 2, 2),
102 | (4, 4, 4, 2016-01-03, 1, 1),
103 | (5, 5, 2, 2016-01-03, 4, 5),
104 | (6, 1, 4, 2016-01-04, 4, 5),
105 | (7, 6, 4, 2016-01-04, 3, 4),
106 | (8, 3, 1, 2016-01-04, 2, 3),
107 | (9, 1, 4, 2016-01-05, 1, 1),
108 | (10, 7, 4, 2016-01-05, 3, 4),
109 | (11, 8, 4, 2016-01-05, 2, 3);
110 |
111 | INSERT INTO order_items (order_id, item_id, product_id, quantity) VALUES

```

```
110 | ( 1, 1, 15, 1),  
111 | ( 1, 2, 13, 2),  
112 | ( 2, 1, 1, 1),  
113 | ( 3, 1, 1, 1),  
114 | ( 4, 1, 4, 2),  
115 | ( 5, 1, 5, 2),  
116 | ( 5, 2, 6, 1),  
117 | ( 6, 1, 9, 1),  
118 | ( 7, 1, 8, 1),  
119 | ( 7, 2, 1, 1),  
120 | ( 7, 3, 4, 2),  
121 | ( 8, 1, 11, 1),  
122 | ( 9, 1, 14, 2),  
123 | (10, 1, 14, 1),  
124 | (11, 1, 7, 1),  
125 | (11, 2, 14, 1),  
126 | (11, 3, 15, 1);
```